

تقرير التسليم النهائي

الطلبات 10 - 6

High-Performance E-Commerce Backend Engine

البند	القيمة
اسم المشروع	محرك معالجة عالي الأداء لمنظومة تجارة إلكترونية
نطاق التقرير	المتطلبات غير الوظيفية 6 إلى 10
التقنيات الأساسية	.NET Web API, Redis, EF Core, SQL Server LocalDB, JMeter
نوع الأدلة	Swagger Screenshots + JMeter Reports + Charts + Design Diagrams
الحالة النهائية	تم تنفيذ المتطلبات مع إثباتات قبل/بعد وقياسات رقمية

هذا التقرير لا يكتفي بسرد ما تم تنفيذه، بل يشرح المشكلة الهندسية، الحلول البديلة التي تم رفضها، سبب اختيار التقنية النهائية، الصعوبات التي واجهتنا، وطريقة التحقق بالأرقام والصور.

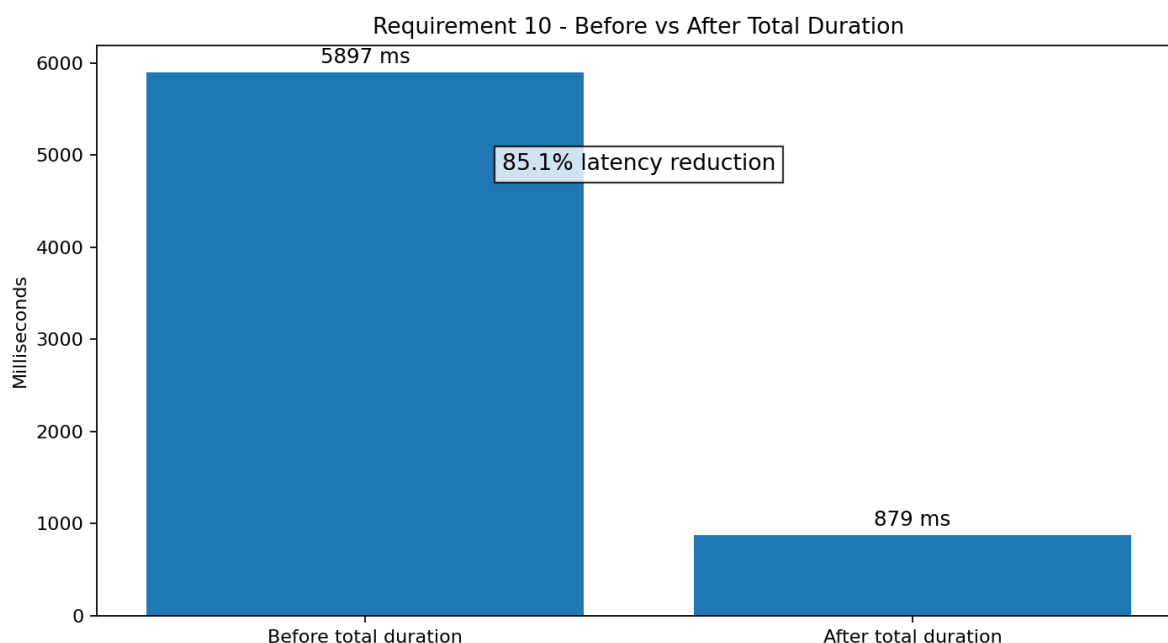
- ملخص تنفيذي للطلبات 6 إلى 10
- المنهجية العامة في التصميم والاختبار
- Requirement 06 - Distributed Caching using Redis
- Requirement 07 - Real Database Optimistic Locking
- Requirement 08 - Real Database ACID Transaction
- Requirement 09 - Stress Testing with 100 Concurrent Users
- Requirement 10 - Bottleneck Analysis & Benchmarking
- مصفوفة الأدلة النهائية والصور المستخدمة
- خطة إعادة الاختبار من JMeter و Swagger
- الخلاصة النهائية

1. الملخص التنفيذي

يغطي هذا التقرير الجزء الثاني من مشروع مادة البرمجة المتوازية، وتحديدًا المتطلبات من 6 إلى 10. الهدف في هذه المرحلة لم يكن إضافة واجهات أو صفحات، بل إثبات أن النظام قادر على العمل تحت ضغط عالٍ مع تقليل زمن الاستجابة، حماية الموارد المشتركة، وضمان سلامة المعاملات.

اعتمدنا على تصميم قابل للقياس: كل متطلب له endpoint واضح في Swagger، وملفات JMeter عند الحاجة، وصور إثبات تظهر النتائج الرقمية. كما تم الحفاظ على بعض النسخ القديمة من الـ endpoints لأنها مفيدة في شرح المشكلة قبل التحسين، بينما تم اعتماد النسخ الأقوى كإثبات نهائي للمتطلبات 7 و 8.

المتطلب	الحل النهائي المعتمد	دليل النجاح الأساسي
06 - Caching	Redis عبر IDistributedCache ونمط Cache-Aside	Cache hits واضحة، Redis configured، ونتائج قبل/بعد
07 - Concurrency Control	SQL Server RowVersion + EF Core Optimistic Locking	10 مستخدمين على مخزون 5، النجاح 5 فقط، finalStock = 0
08 - ACID Transaction	EF Core Database Transaction + Serializable Isolation	Rollback حقيقي عند الفشل و Commit كامل عند النجاح
09 - Stress Testing	JMeter + endpoint داخلي لـ 100 مستخدم	1101 samples، Error = 0.00%، Throughput = 88.8/sec
10 - Benchmarking	Benchmark داخلي قبل/بعد + تحديد Bottleneck	قبل 5897ms، بعد 879ms، تحسن 85.1%



مخطط ملخص: مقارنة زمن التنفيذ الكلي قبل وبعد التحسين في الطلب 10.

2. المنهجية العامة في التصميم والاختبار

تم اعتماد مبدأين أساسيين في التنفيذ: الأول هو فصل المسؤوليات بين Endpoints و Services و Models/Entities ، والثاني هو توفير دليل قابل للتشغيل لكل متطلب. لذلك تظهر في Swagger endpoints مباشرة للاختبار، وتوجد ملفات JMeter للضغط الخارجي، كما توجد وثائق Markdown داخل مجلد Docs لكل متطلب.

2.1 لماذا اعتمدنا Swagger و JMeter معاً؟

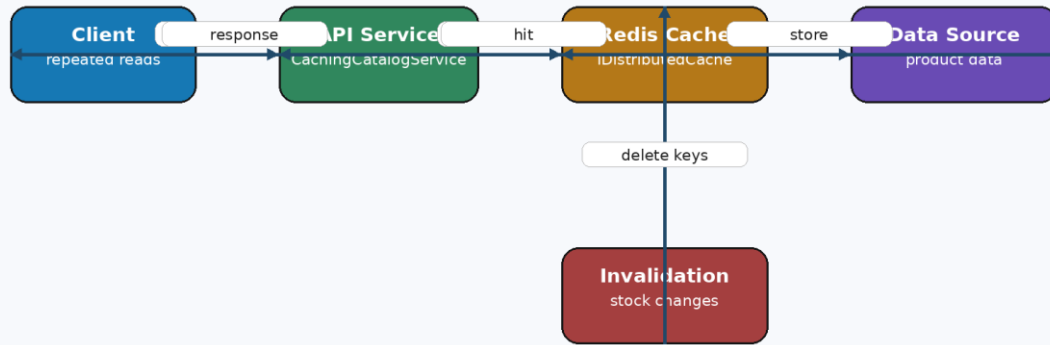
Swagger مناسب لإثبات السيناريوهات الوظيفية والمنطقية بشكل واضح أمام الدكتور reset: ثم before/after ثم قراءة metrics. أما JMeter فهو مناسب لإثبات الضغط الحقيقي من خارج التطبيق، خصوصاً في الطلب 9 حيث يجب إظهار 100 مستخدم متزامن وأرقام مثل Max, Average, Throughput, Error %.

2.2 البنية العامة للأدلة

نوع الدليل	استخدامه في التقرير
Swagger screenshots	إثبات صحة السلوك النهائي لكل endpoint
JMeter Summary/Aggregate	إثبات الضغط، عدد العينات، نسبة الأخطاء، والthroughput
Charts/SVG	تقديم بصري واضح للموارد والاختناقات
Docs داخل المشروع	شرح التصميم، الملفات، والسبب الهندسي وراء كل حل
SQL LocalDB وMigrations	إثبات أن الطالبين 7 و 8 ليسا simulation فقط، بل يعتمدان على قاعدة بيانات حقيقية

Requirement 06 - Distributed Caching Strategy .3

Requirement 06 - Cache-Aside with Redis



مخطط معماري: نمط Cache-Aside مع Redis و IDistributedCache.

3.1 المشكلة الهندسية

في نظام التجارة الإلكترونية تتكرر عمليات قراءة نفس البيانات بشكل كبير: تفاصيل المنتج، المنتجات الأكثر طلبًا، ولقطات المخزون السريعة. بدون Cache كل هذه الطلبات تضغط على مصدر البيانات، حتى لو كانت النتيجة نفسها مطلوبة مئات المرات خلال ثوانٍ قليلة.

المشكلة ليست فقط في زمن الاستجابة، بل في قابلية التوسع. قاعدة البيانات يجب أن تُستخدم للبيانات التي تحتاج دقة وكتابة، لا أن تتحول إلى نقطة اختناق بسبب قراءات مكررة قابلة للتخزين المؤقت.

3.2 لماذا اخترنا Redis و Cache-Aside ؟

الخيار	لماذا لم يكن كافياً؟	القرار
بدون Cache	بسيط لكنه يكرر الاستعلامات ويزيد الضغط	مرفوض
In-Memory Cache	سريع لكنه محلي داخل instance واحدة ولا يناسب توزيع الأحمال	غير كافٍ
CDN	ممتاز للملفات الثابتة، وليس مناسباً مباشرة لمخزون ومنتجات متغيرة	غير مناسب لهذا المتطلب
Redis Distributed Cache	مشارك بين أكثر من server ويدعم TTL و invalidation	تم اختياره

تم استخدام IDistributedCache بدل ربط الكود مباشرة بمكتبة Redis ، لأن ذلك يعطي مرونة: يمكن تشغيل النظام محليًا، ثم استخدام Redis فعليًا بالإعدادات دون تغيير منطق الخدمات.

3.3 طريقة التنفيذ

- تطبيق Cache-Aside اقرأ من Redis أولاً، وإذا لم توجد القيمة اقرأ من مصدر البيانات ثم خزن النتيجة.
- تحديد TTL مختلف حسب حساسية البيانات: المنتجات الأكثر طلباً أطول عمراً، أما stock snapshot فعمره قصير.
- إضافة invalidation عند تعديل المخزون لتجنب stale data.
- إضافة metrics لعدد cache hits و cache misses و direct database reads.

العنصر المخزن	المفتاح المستخدم	سبب الكاش
Product details	products:details:{productId}	قراءة متكررة لنفس المنتج
Popular products	products:popular:top:{count}	قائمة كثيرة الطلب ولا تتغير بكل عملية
Stock snapshot	inventory:stock:{productId}	قراءة سريعة مع TTL قصير بسبب حساسية المخزون

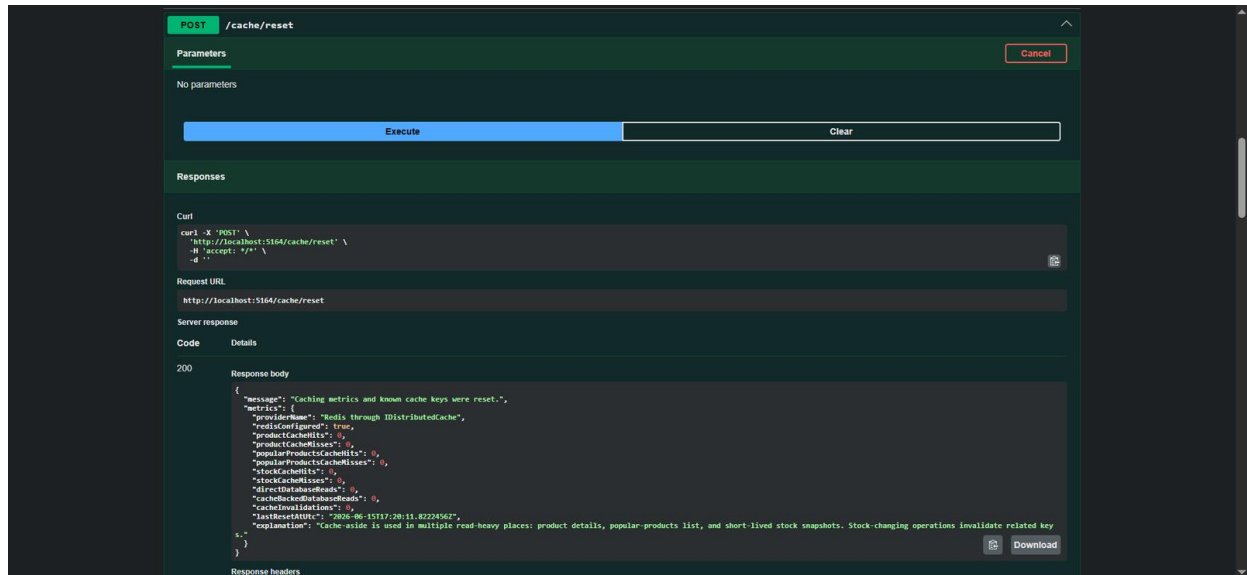
3.4 المشاكل التي واجهتنا

- التأكد أن Redis يعمل فعلياً وليس مجرد إعدادات داخل appsettings.
- اختيار مدة تخزين مناسبة للمخزون حتى لا نعرض بيانات قديمة.
- الحاجة إلى cache invalidation عند عمليات تؤثر على المنتج أو المخزون.
- الحفاظ على fallback عند عدم توفر Redis أثناء التجربة المحلية.

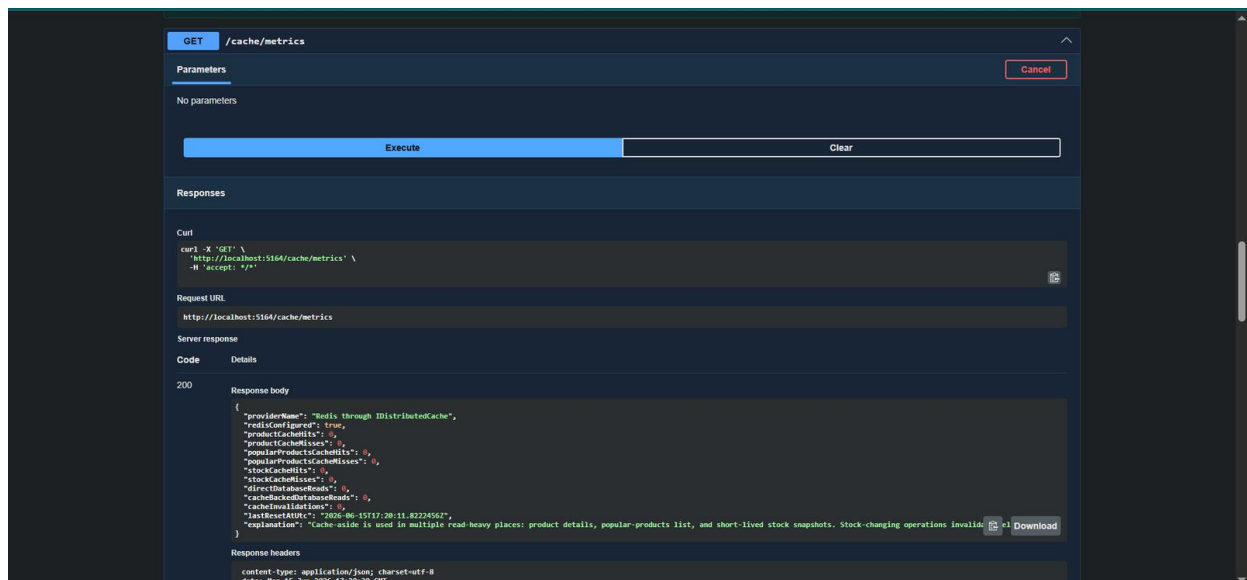
3.5 نتائج الاختبار

في الاختبار النهائي ظهر أن Redis configured = true وأن provider هو Redis through IDistributedCache. بعد تنفيذ demo after ، تم تنفيذ 30 قراءة للمنتجات و 10 للمنتجات الشائعة و 10 للمخزون. ظهرت hits عالية بعد أول miss لكل مفتاح: productCacheHits = 29، popularProductsCacheHits = 9، stockCacheHits = 9. هذا يثبت أن الطلبات المتكررة لم تعد تضرب مصدر البيانات مباشرة.

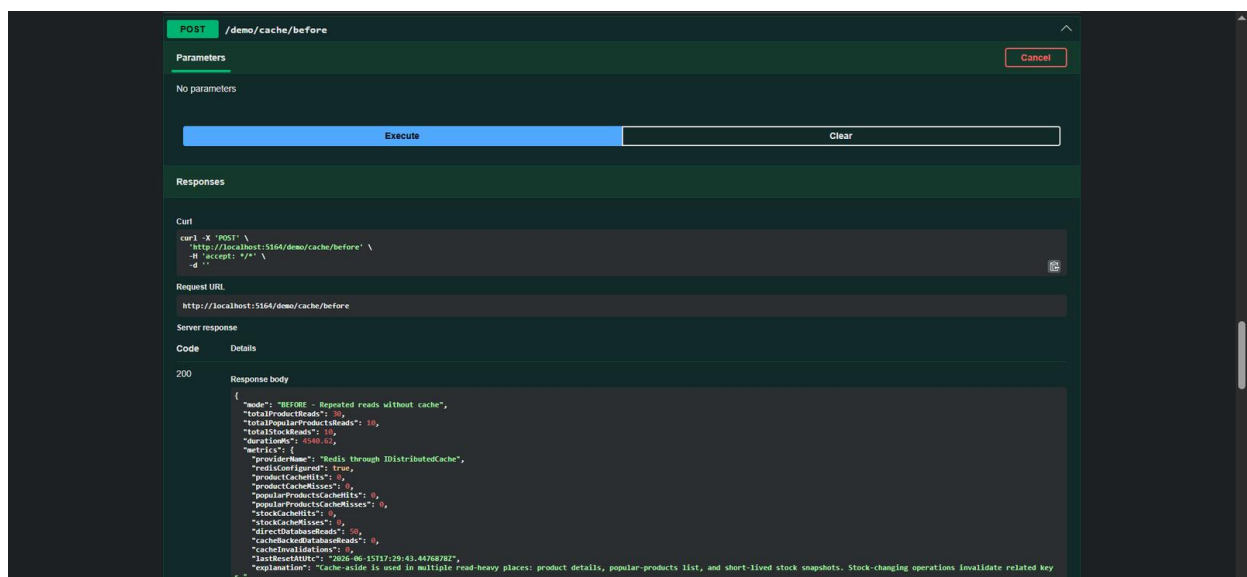
المؤشر	القيمة من الاختبار	الدلالة
RedisConfigured	true	Redis مفعّل فعلياً
ProductCacheHits	29	أغلب قراءات المنتج جاءت من الكاش
PopularProductsCacheHits	9	تخزين قائمة المنتجات الأكثر طلباً
StockCacheHits	9	لقطات المخزون استقادت من TTL قصير
CacheInvalidations	0 في الاختبار المعروض	لم يحدث تعديل يستدعي حذف الكاش أثناء هذه الجولة



إثبات إعادة ضبط الكاش قبل الاختبار.



إثبات أن Redis configured وأن metrics بدأت من حالة نظيفة.



نتيجة قبل التحسين: قراءة مباشرة بدون الاستفادة من الكاش.

Execute

Clear

Responses

Curl

curl -X 'POST' \n'http://localhost:5164/demo/cache/after' \n-H 'accept: */*' \n-d ''

Request URL

http://localhost:5164/demo/cache/after

Server response

Code

Details

200

Response body

{\n "mode": "AFTER - Repeated reads with cache-aside",\n "totalProductReads": 30,\n "totalPopularProductReads": 10,\n "totalStockReads": 10,\n "durationMs": 579.10,\n "metrics": {\n "providerName": "Redis through IDistributedCache",\n "redisConfigured": true,\n "productCacheHits": 20,\n "productCacheMisses": 10,\n "popularProductsCacheHits": 10,\n "popularProductsCacheMisses": 0,\n "stockCacheHits": 10,\n "stockCacheMisses": 0,\n "directDatabaseReads": 0,\n "cachedDatabaseReads": 0,\n "cacheInvalidations": 0,\n "lastResponseStatus": "200: 46-15717:30:41.42000492",\n "explanation": "Cache-aside is used in multiple read-heavy places: product details, popular-products list, and short-lived stock snapshots. Stock-changing operations invalidate related key\n },\n "explanation": "Only the first request for each cache key missed. Repeated requests were served from cache." \n}

Download

Response headers

content-type: application/json; charset=utf-8\n date: Mon, 15 Jun 2026 17:30:41 GMT\n server: Kestrel\n transfer-encoding: chunked

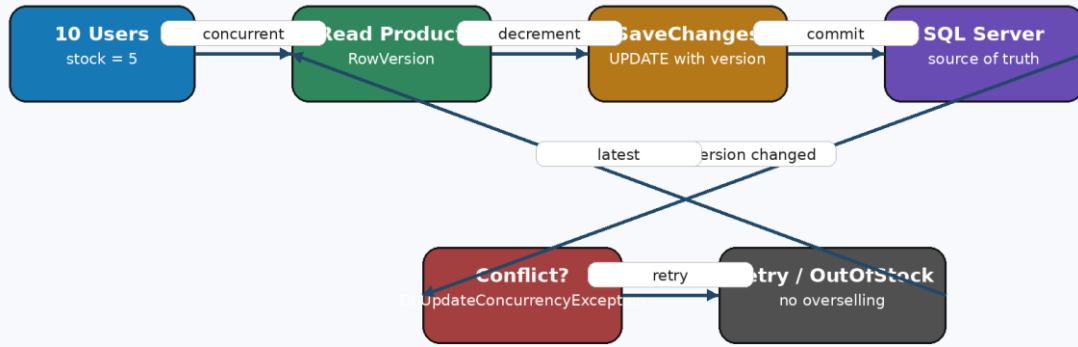
Response

نتيجة after: cache hits واضحة بعد تكرار القراءات.

Requirement 07 - Concurrency Control / Optimistic .4

Locking

Requirement 07 - SQL RowVersion Optimistic Locking



مخطط معماري: آلية Optimistic Locking باستخدام RowVersion في SQL Server.

4.1 المشكلة الهندسية

المخزون مورد مشترك حساس. إذا قرأ 10 مستخدمين نفس كمية المخزون في نفس اللحظة، ثم حاول كل واحد منهم خصم وحدة، قد يحدث overselling إذا لم تكن هناك حماية. في مشروع تجارة إلكترونية هذا خطأ خطير لأنه يعني بيع منتجات غير موجودة فعليًا.

4.2 لماذا لم نكتفِ بالنسخة القديمة Distributed Lock ؟

في المشروع توجد نسخة Redis Distributed Lock للكوبونات وعمليات الدفع. هذه مفيدة جدًا كنموذج حماية موارد موزعة، لكنها لا تطابق نص الطلب حرفيًا لأن الدكتور طالب تطبيق Optimistic أو Pessimistic Locking عند تحديث كميات المخزون الحساسة. لذلك أضفنا نسخة أقوى تستخدم قاعدة بيانات حقيقية و RowVersion.

4.3 البدائل التقنية ولماذا اخترنا Optimistic Locking

الحل	المزايا	سبب عدم الاعتماد عليه كحل نهائي
lock محلي في C#	بسيط ويمنع race داخل نفس process	لا يحمي عند وجود أكثر من server
Redis Distributed Lock	مناسب للموارد الموزعة مثل coupon/payment callback	لا يثبت RowVersion أو DB-level inventory concurrency
Pessimistic DB Lock	يحجز الصف ويمنع المنافسة	قد يقلل التوازي ويزيد الانتظار تحت الضغط
Optimistic RowVersion	يكشف التعارض عند الحفظ ويعيد المحاولة	تم اختياره لأنه يثبت حماية مخزون حقيقية مع توازي جيد

4.4 طريقة التنفيذ

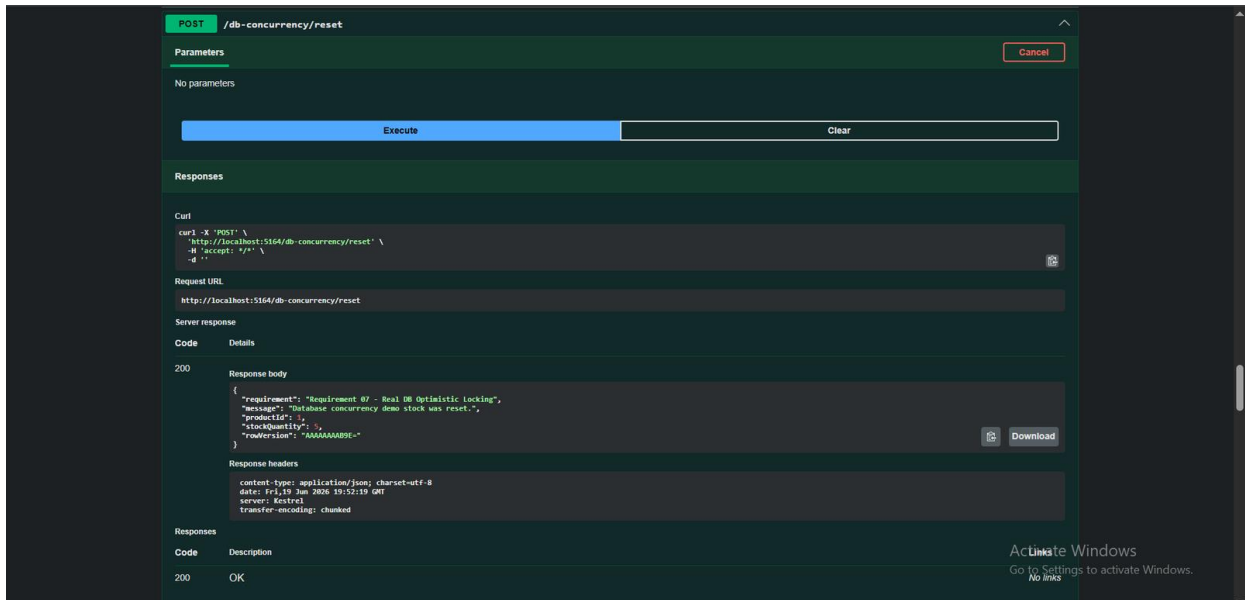
- إضافة ProductEntity وفيه RowVersion من نوع byte[].
- تهيئة RowVersion في DbContext باستخدام IsRowVersion().
- تشغيل 10 محاولات شراء متزامنة على stock = 5.
- كل attempt يقرأ النسخة الحالية ويحاول SaveChangesAsync.
- عند تغير النسخة يتم رمي DbUpdateConcurrencyException ثم retry.
- إذا انتهى المخزون يتم إرجاع OutOfStock بدل السماح ببيع زائد.

4.5 المشاكل التي واجهتنا

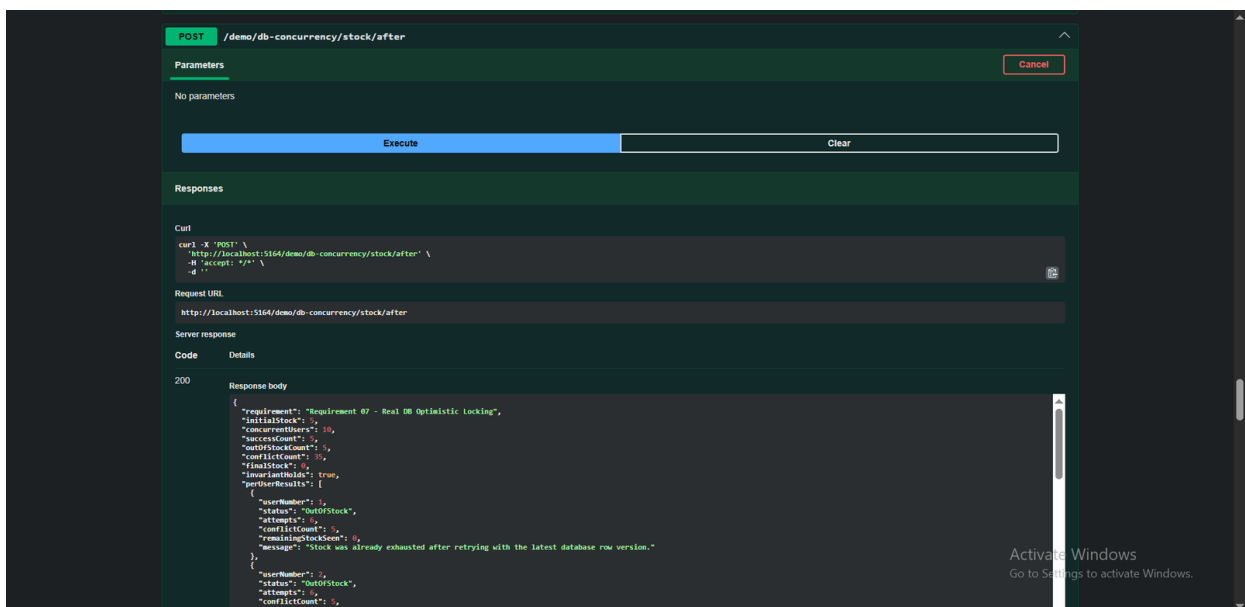
- النسخة الأولى من الطلب 7 كانت قوية لكنها لا تطابق النص حرفيًا، لذلك أضفنا Real DB Optimistic Locking.
- ظهرت مشكلة MSSQLLocalDB instance LocalDB: غير موجود، وتم حلها بإنشاء وتشغيل instance.
- ظهر تعارض في تسجيل DbContext و DbContextFactory ، وتم اعتماد IDbContextFactory لكل محاولة متزامنة.
- تم إنشاء EF Core Migration باسم AddRealDatabaseForReq07Req08 لضمان أن قاعدة البيانات تحتوي RowVersion والجدول المطلوبة.

4.6 نتائج الاختبار

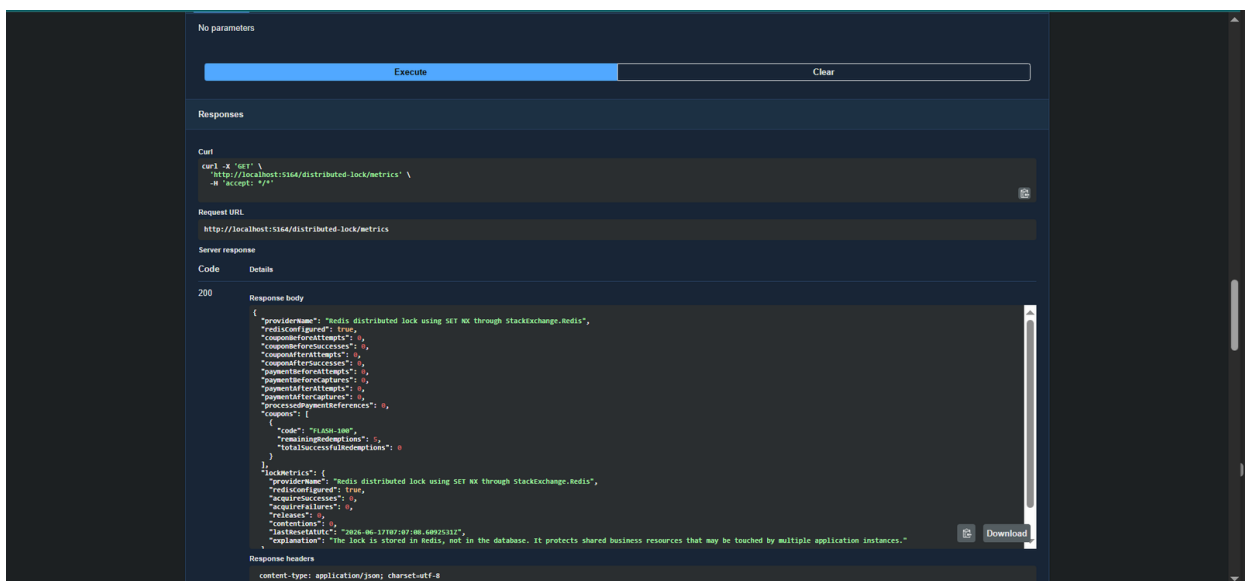
المؤشر	القيمة	الدلالة
InitialStock	5	المخزون الابتدائي محدود
ConcurrentUsers	10	عدد محاولات الشراء المتزامنة
SuccessCount	5	لا يمكن أن يزيد عدد النجاحات عن المخزون
OutOfStockCount	5	الطلبات الزائدة فشلت تجاريًا بشكل صحيح
ConflictCount	35	EF Core كشف تعارضات حقيقية في RowVersion
FinalStock	0	لا يوجد مخزون سالب
InvariantHolds	true	القاعدة الحسابية محققة بالكامل



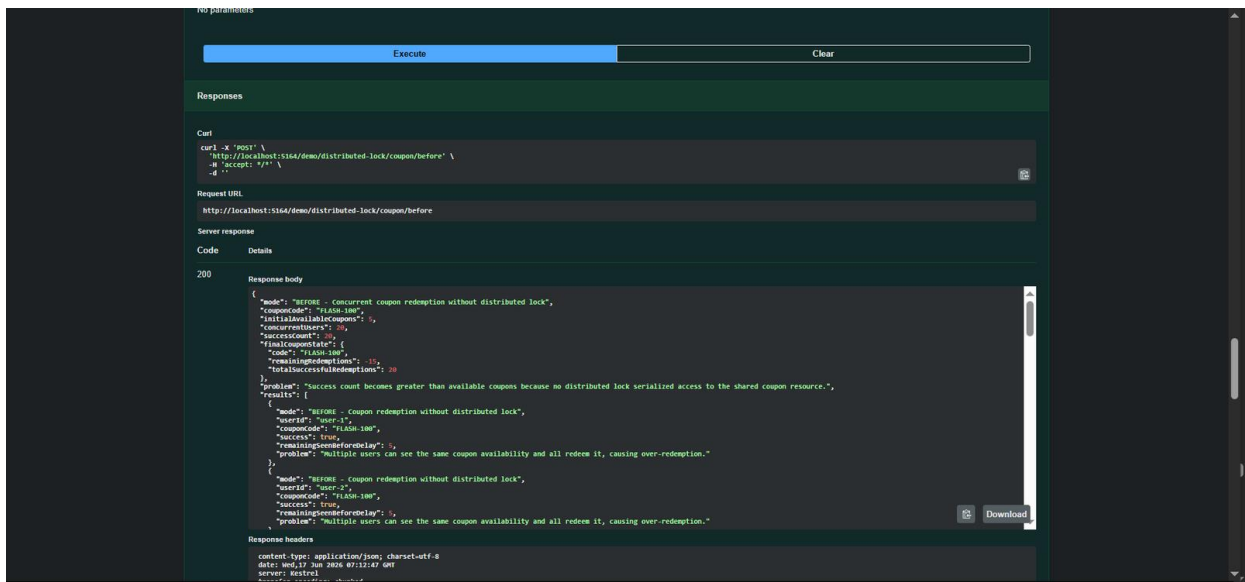
إعادة ضبط مخزون المنتج إلى 5 قبل اختبار. optimistic locking



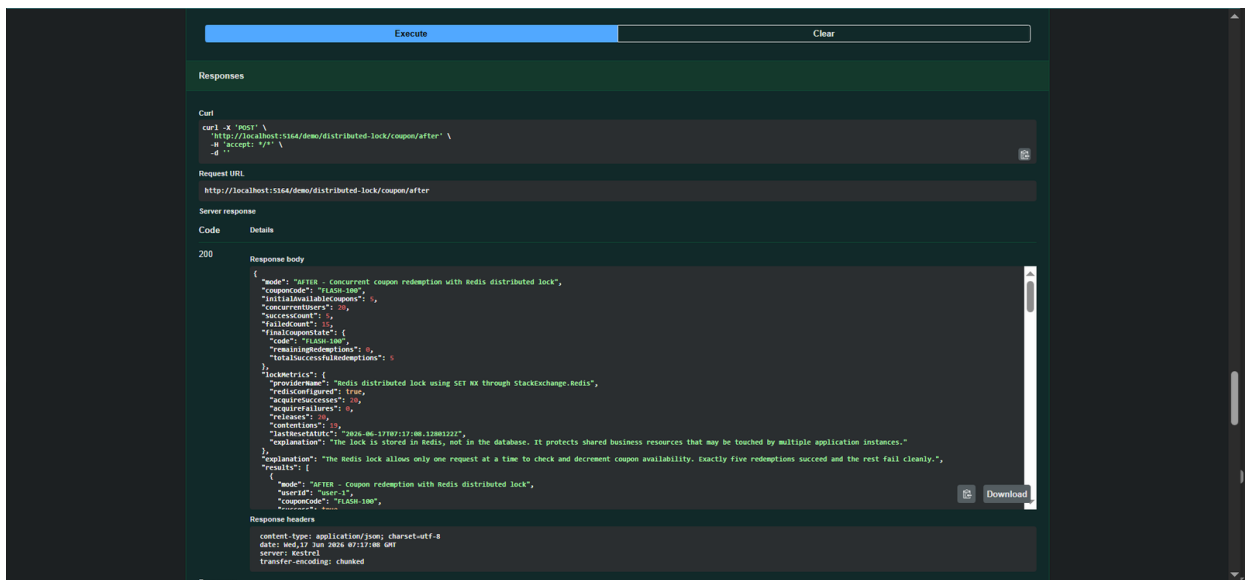
نتيجة: Real DB Optimistic Locking نجاح 5 فقط من أصل 10 ومحافظة على. finalStock = 0



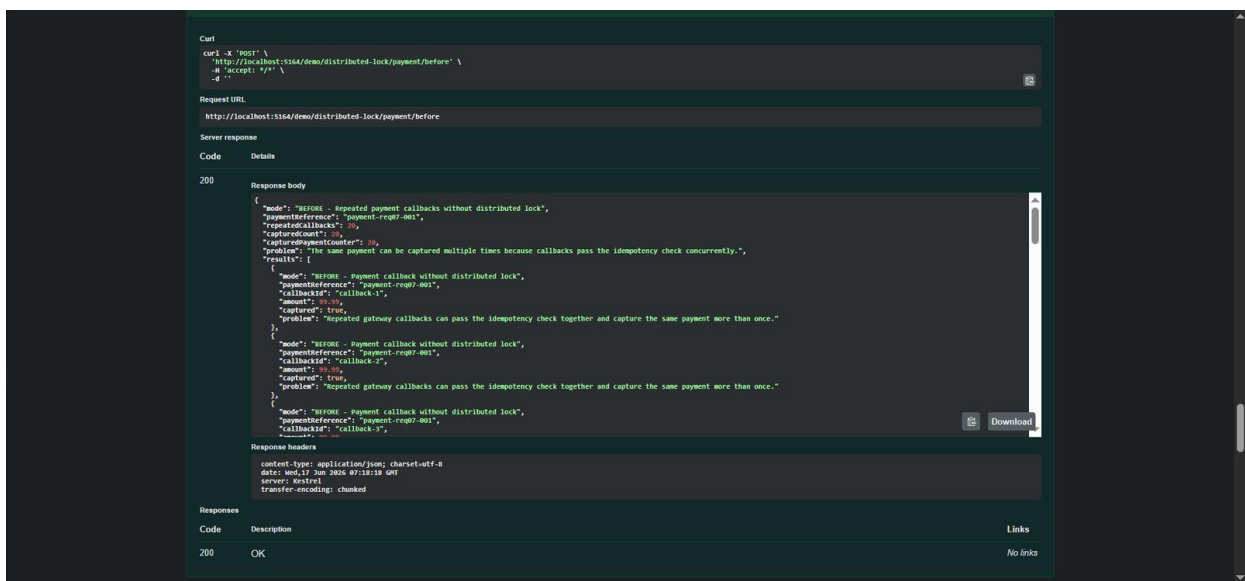
دليل إضافي metrics: للنسخة القديمة من Redis Distributed Lock



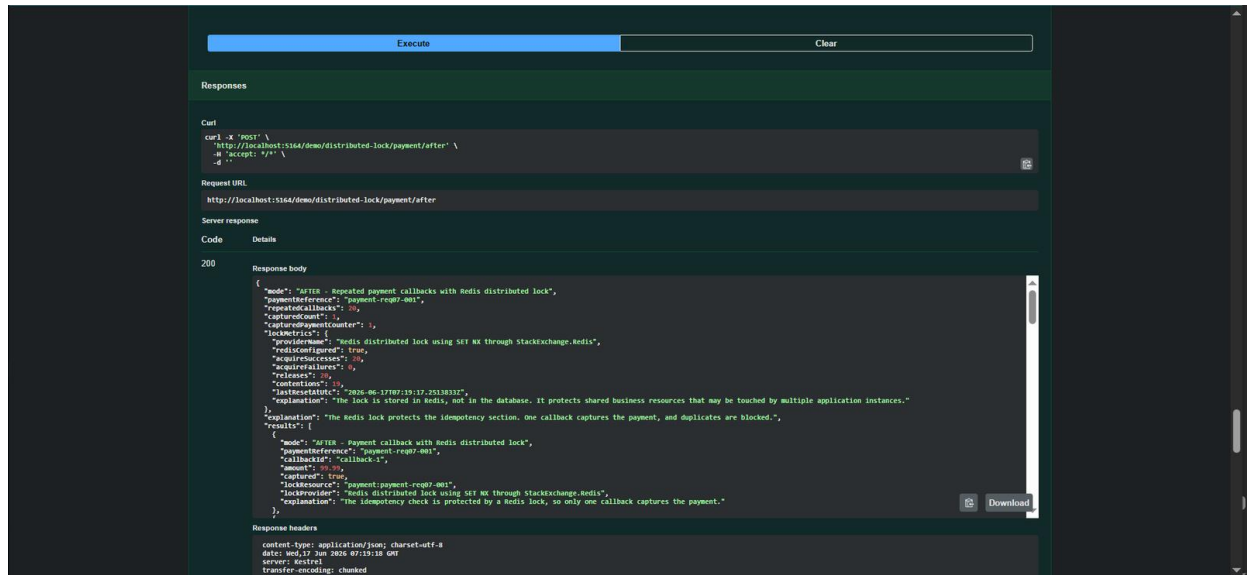
قبل Redis Lock: مشكلة تنافس على كوبون محدود الاستخدام.



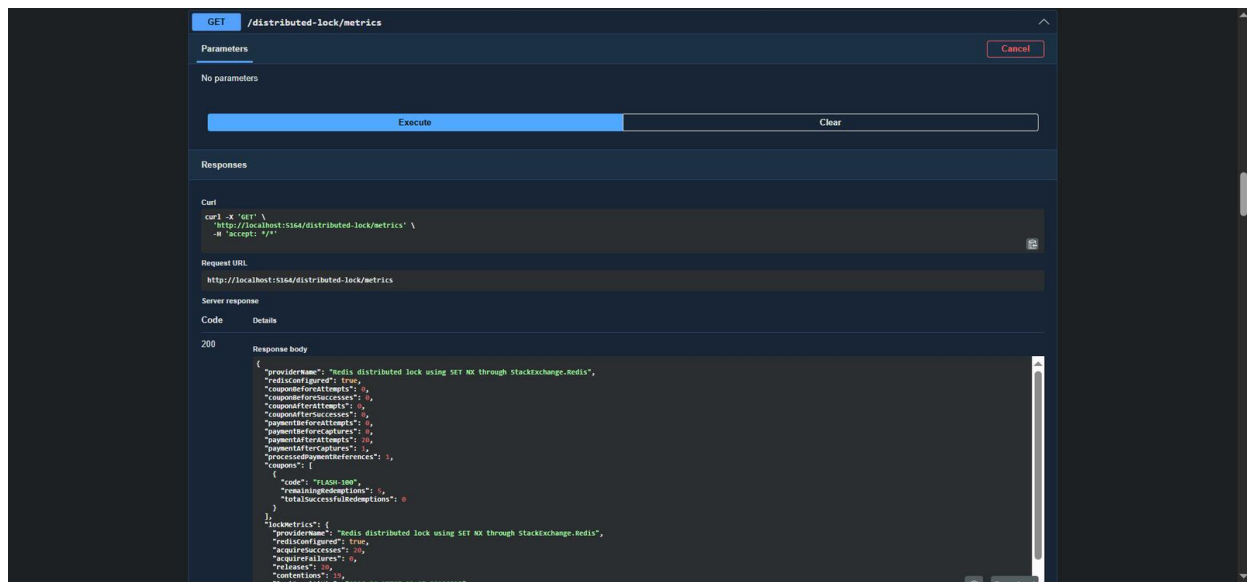
بعد Redis Lock: حماية الكوبون من الاستهلاك الزائد.



قبل الحماية :احتمال تكرار معالجة payment callback.

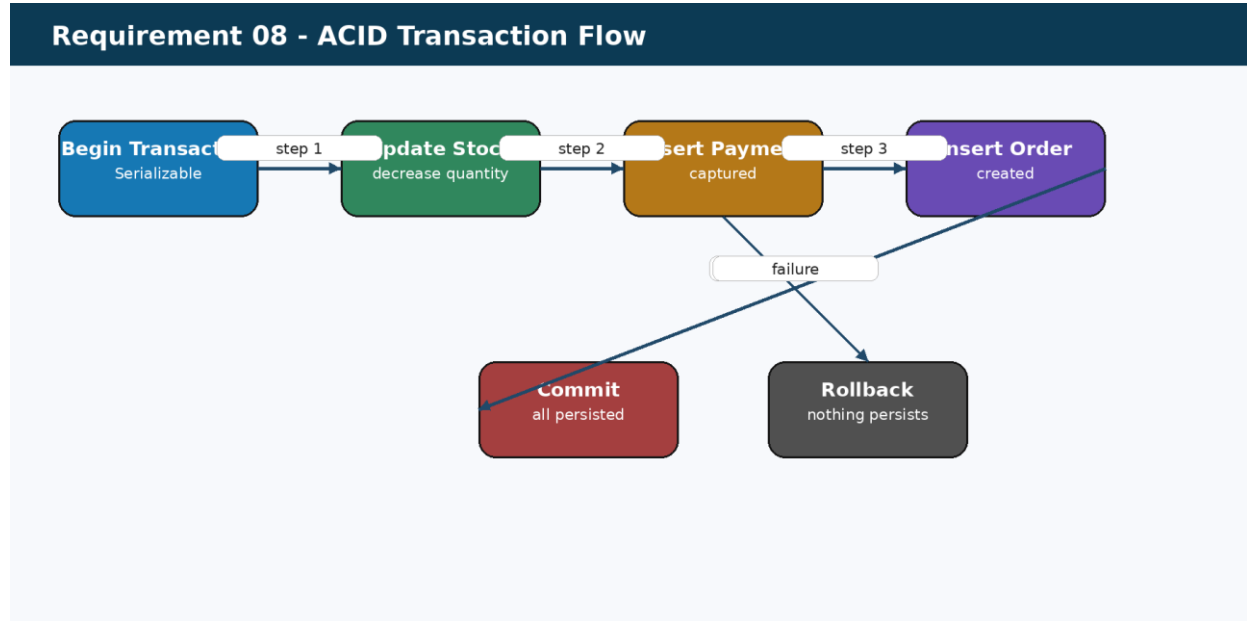


بعد الحماية :منع duplicate payment capture.



المؤشرات النهائية لسيناريوهات distributed lock كإضافة داعم.

Requirement 08 - ACID / Transaction Integrity .5



مخطط معماري transaction: حقيقة تجمع stock update و payment و order creation.

5.1 المشكلة الهندسية

عملية checkout ليست خطوة واحدة. هي عملية مركبة: دفع، خصم مخزون، وإنشاء طلب. الخطر أن تنجح خطوة وتفشل أخرى، مثل تسجيل الدفع دون إنشاء الطلب، أو خصم المخزون دون إتمام العملية. هذا يكسر Atomicity و Consistency.

5.2 لماذا احتجنا قاعدة بيانات حقيقية؟

النسخة القديمة كانت مفيدة لشرح فكرة rollback على مستوى التطبيق، لكنها لا تثبت ACID فعلي داخل قاعدة البيانات. لذلك أضفنا تنفيذًا جديدًا باستخدام EF Core Transaction على SQL Server LocalDB، بحيث يتم commit أو rollback لكل التعديلات معًا.

5.3 البدائل وسبب اختيار EF Core Transaction

الخيار	متى يكون مناسبًا؟	سبب القرار
Manual rollback in memory	تعليمي فقط أو simulation	لا يثبت ACID داخل قاعدة بيانات
Compensation/Saga	أنظمة موزعة متعددة الخدمات	أكثر تعقيدًا من متطلب المشروع
Distributed Transaction	عند وجود أكثر من DB/Service	غير ضروري لأن العمليات داخل قاعدة واحدة
EF Core Transaction	عملية مركبة داخل SQL Server	تم اختياره لأنه واضح وقابل للإثبات

5.4 طريقة التنفيذ

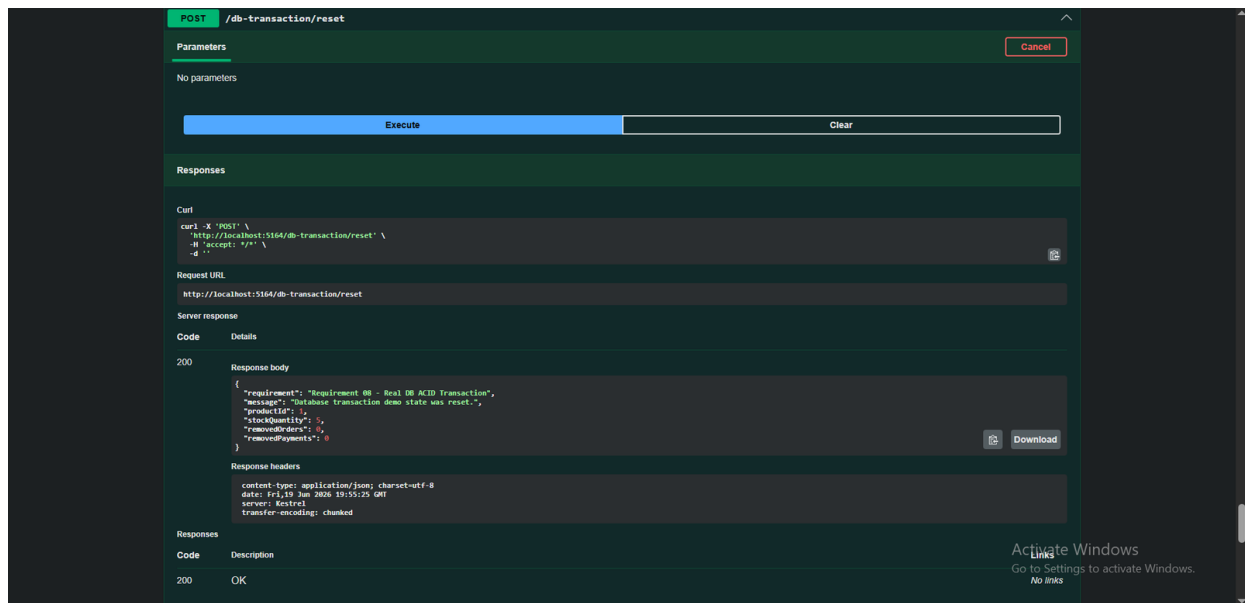
- بدء transaction بعزل Serializable.
- قراءة المنتج والتحقق من المخزون.
- تحديث المخزون داخل نفس transaction.
- إدراج PaymentEntity.
- محاكاة failure بعد الدفع لإثبات rollback.
- إدراج OrderEntity عند عدم وجود failure.
- Commit عند النجاح، Rollback عند أي exception.

5.5 المشاكل التي واجهتنا

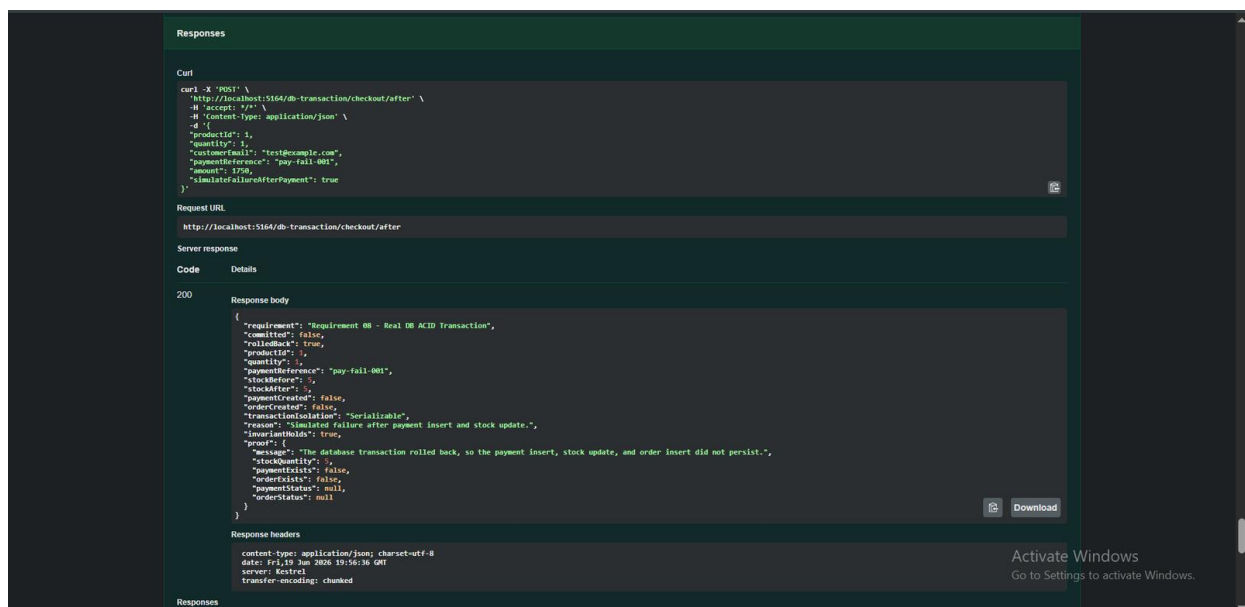
- ضرورة منع duplicate paymentReference باستخدام unique index حتى لا تُسجل نفس عملية الدفع مرتين.
- الحاجة إلى فصل DbContext لكل عملية لتجنب مشاكل lifetime.
- التحقق من أن rollback لا يترك Payment أو Order في القاعدة.
- ضرورة إعادة reset أو تغيير paymentReference عند تكرار الاختبار من Swagger.

5.6 نتائج الاختبار

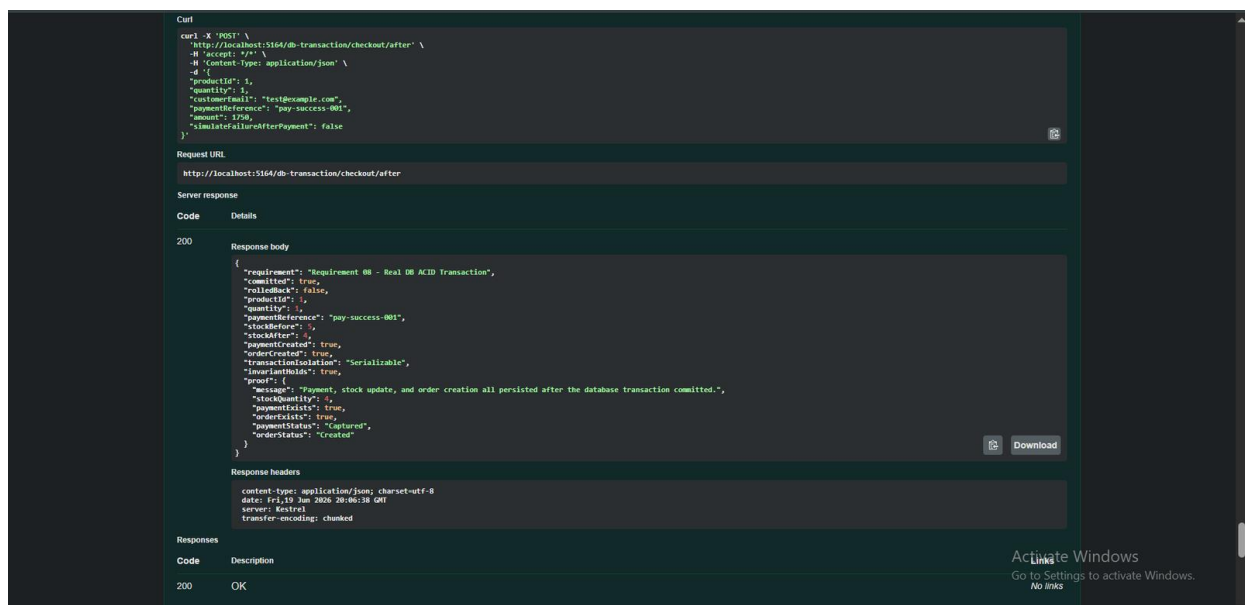
النتيجة	القيم الأساسية	السيناريو
نجاح rollback كامل	committed=false, rolledBack=true, stockBefore=5, stockAfter=5, paymentCreated=false, orderCreated=false	Rollback
نجاح العملية المركبة بالكامل	committed=true, rolledBack=false, stockBefore=5, stockAfter=4, paymentCreated=true, orderCreated=true	Commit
تقليل احتمالات interleaving غير آمن أثناء checkout	Serializable	Isolation



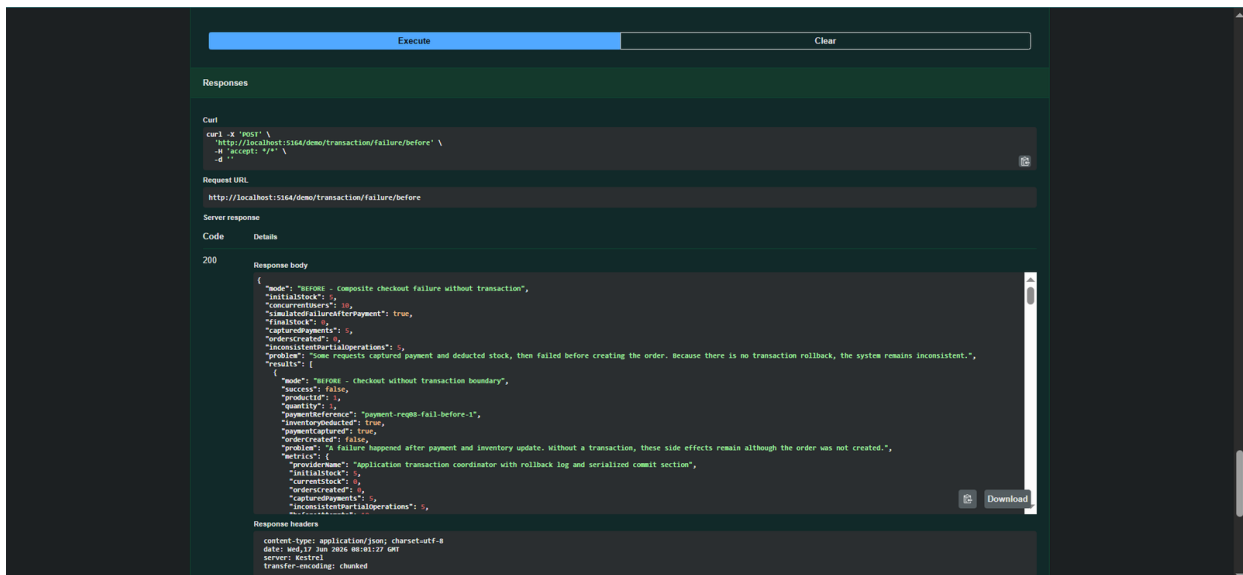
إعادة ضبط قاعدة البيانات للطلب: 8 المخزون 5 وحذف orders/payments التجريبية.



اختبار: rollback فشل متعمد بعد الدفع، والنتيجة عدم بقاء payment أو order.



اختبار commit: إنشاء payment و order وتحديث stock من 5 إلى 4.



Execute Clear

Responses

Curl

```
curl -X 'POST' \
  http://localhost:5164/demo/transaction/failure/before/ \
  -H 'accept: */*' \
  -d ''
```

Request URL

http://localhost:5164/demo/transaction/failure/before

Server response

Code Details

200

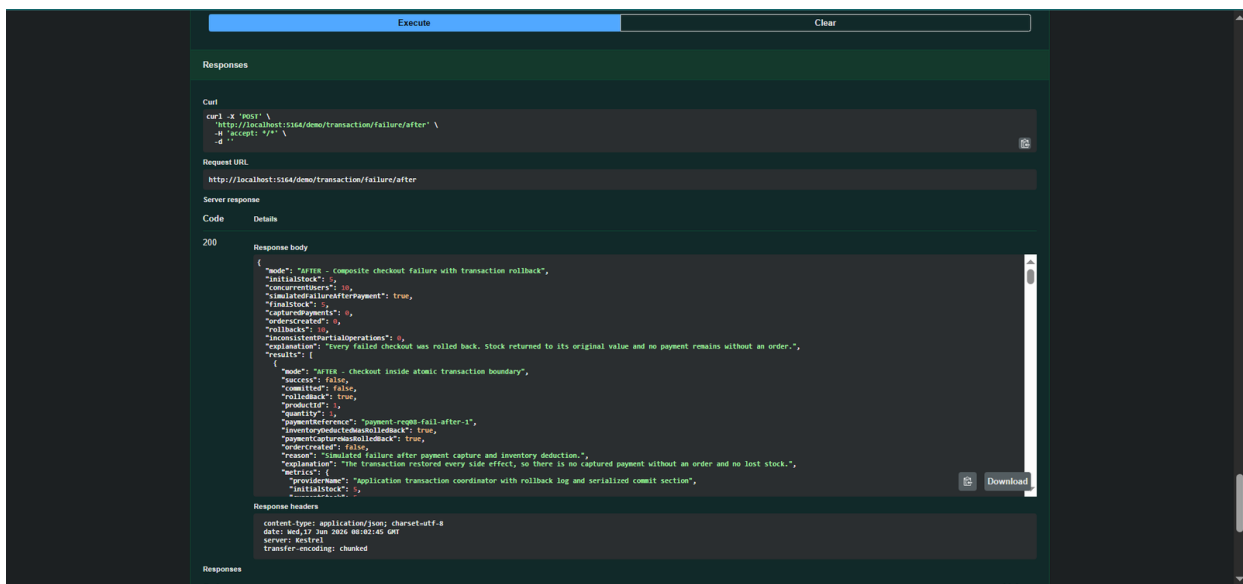
Response body

```
{
  "mode": "BEFORE - Composite checkout failure without transaction",
  "initialStock": 5,
  "concurrentOrders": 0,
  "simulatedFailureAfterPayment": true,
  "finalStock": 5,
  "capturePayments": 0,
  "orderCreated": 0,
  "inconsistentPartialOperations": 0,
  "problems": "Some requests captured payment and deducted stock, then failed before creating the order. Because there is no transaction rollback, the system remains inconsistent.",
  "results": [
    {
      "mode": "BEFORE - Checkout without transaction boundary",
      "success": false,
      "commit": false,
      "quantity": 1,
      "product": "i",
      "paymentReference": "payment-reqns-fail-before-1",
      "inventoryDeducted": true,
      "paymentCaptured": true,
      "orderCreated": false,
      "rollback": false,
      "inconsistentPartialOperations": 0,
      "problems": "A failure happened after payment and inventory update. Without a transaction, these side effects remain although the order was not created.",
      "metrics": {
        "providerName": "Application transaction coordinator with rollback log and serialized commit section",
        "initialStock": 5,
        "currentStock": 5,
        "orderCreated": 0,
        "capturePayments": 0,
        "inconsistentPartialOperations": 0
      }
    }
  ]
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 17 Jun 2020 08:01:11 GMT
server: Kestrel
transfer-encoding: chunked
```

النسخة القديمة قبل التحسين: توضيح مشكلة partial failure.



Execute Clear

Responses

Curl

```
curl -X 'POST' \
  http://localhost:5164/demo/transaction/failure/after/ \
  -H 'accept: */*' \
  -d ''
```

Request URL

http://localhost:5164/demo/transaction/failure/after

Server response

Code Details

200

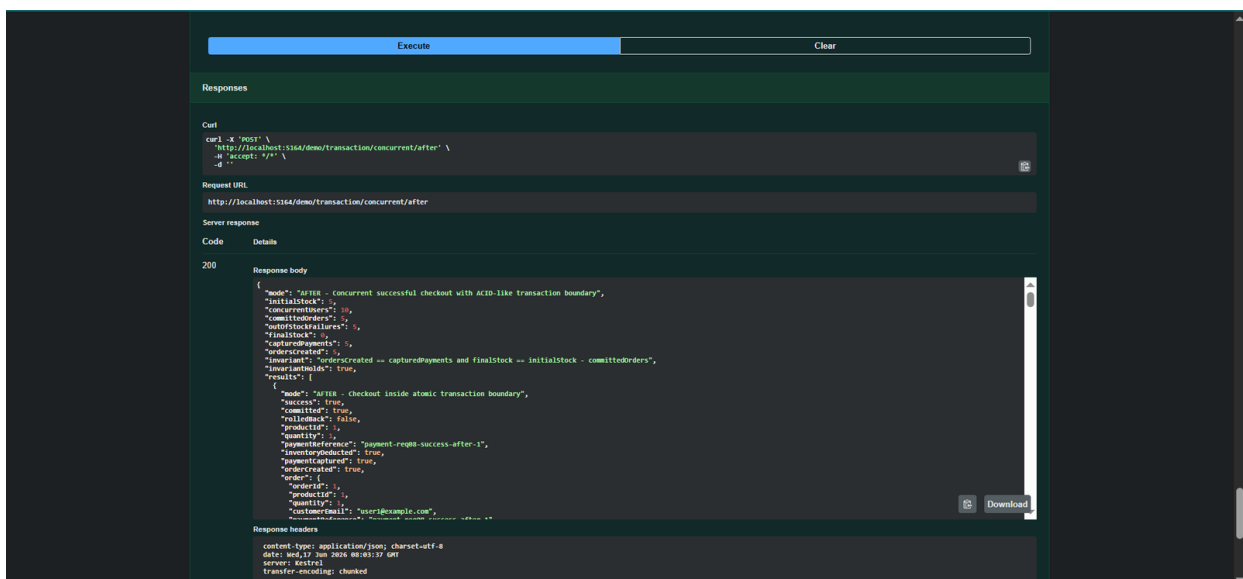
Response body

```
{
  "mode": "AFTER - Composite checkout failure with transaction rollback",
  "initialStock": 5,
  "concurrentOrders": 0,
  "simulatedFailureAfterPayment": true,
  "finalStock": 5,
  "capturePayments": 0,
  "orderCreated": 0,
  "rollback": true,
  "inconsistentPartialOperations": 0,
  "explanation": "Every failed checkout was rolled back. Stock returned to its original value and no payment remains without an order.",
  "results": [
    {
      "mode": "AFTER - Checkout inside atomic transaction boundary",
      "success": false,
      "commit": false,
      "quantity": 1,
      "product": "i",
      "paymentReference": "payment-reqns-fail-after-1",
      "inventoryDeductedAndRolledBack": true,
      "paymentCapturedAndRolledBack": true,
      "orderCreated": false,
      "rollback": true,
      "inconsistentPartialOperations": 0,
      "problems": "Simulated failure after payment capture and inventory deduction.",
      "explanation": "The transaction restored every side effect, so there is no captured payment without an order and no lost stock.",
      "metrics": {
        "providerName": "Application transaction coordinator with rollback log and serialized commit section",
        "initialStock": 5,
        "currentStock": 5,
        "orderCreated": 0,
        "capturePayments": 0,
        "inconsistentPartialOperations": 0
      }
    }
  ]
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 17 Jun 2020 08:02:45 GMT
server: Kestrel
transfer-encoding: chunked
```

النسخة القديمة بعد التحسين rollback: منطقي على مستوى التطبيق.



Execute Clear

Responses

Curl

```
curl -X 'POST' \
  http://localhost:5164/demo/transaction/concurrent/after/ \
  -H 'accept: */*' \
  -d ''
```

Request URL

http://localhost:5164/demo/transaction/concurrent/after

Server response

Code Details

200

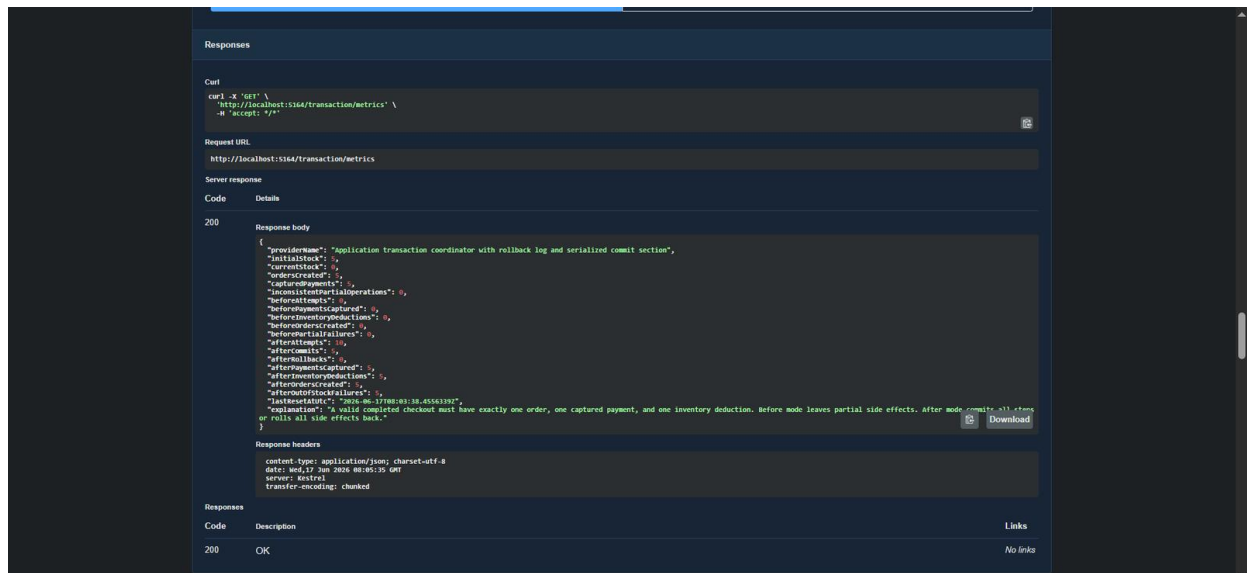
Response body

```
{
  "mode": "AFTER - Concurrent successful checkout with ACID-like transaction boundary",
  "initialStock": 5,
  "concurrentOrders": 0,
  "commitOrders": 0,
  "outOfStockFailures": 0,
  "finalStock": 4,
  "capturePayments": 0,
  "orderCreated": 0,
  "invariant": "ordersCreated == capturePayments and finalStock == initialStock - committedOrders",
  "inconsistentPartialOperations": 0,
  "results": [
    {
      "mode": "AFTER - Checkout inside atomic transaction boundary",
      "success": true,
      "commit": true,
      "quantity": 1,
      "product": "i",
      "paymentReference": "payment-reqns-success-after-1",
      "inventoryDeducted": true,
      "paymentCaptured": true,
      "orderCreated": true,
      "rollback": false,
      "inconsistentPartialOperations": 0,
      "problems": "Simulated failure after payment capture and inventory deduction.",
      "explanation": "The transaction restored every side effect, so there is no captured payment without an order and no lost stock.",
      "metrics": {
        "providerName": "Application transaction coordinator with rollback log and serialized commit section",
        "initialStock": 5,
        "currentStock": 4,
        "orderCreated": 1,
        "capturePayments": 1,
        "inconsistentPartialOperations": 0
      }
    }
  ]
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 17 Jun 2020 08:03:17 GMT
server: Kestrel
transfer-encoding: chunked
```

اختبار concurrent checkout في النسخة القديمة كدليل إضافي.



The screenshot displays a REST client interface with the following details:

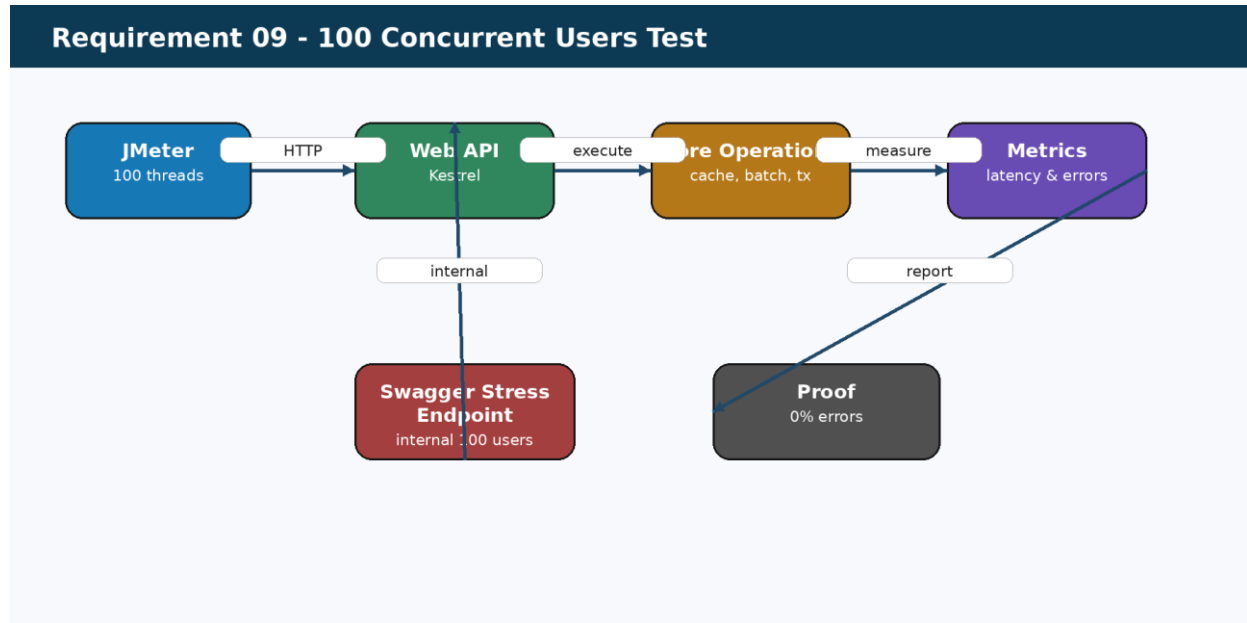
- Method:** GET
- URL:** http://localhost:5164/transaction/metrics
- Status:** 200
- Response Headers:**
 - content-type: application/json; charset=utf-8
 - date: Wed, 27 Nov 2019 01:39:05 GMT
 - server: Kestrel
 - transfer-encoding: chunked
- Response Body (JSON):**

```
{
  "providername": "Application transaction coordinator with rollback log and serialized commit section",
  "initialstock": 0,
  "currentstock": 0,
  "ordercreated": 0,
  "capturepayment": 0,
  "inconsistentinitialoperations": 0,
  "beforecommit": 0,
  "beforepaymentcapture": 0,
  "beforeinventorydeductions": 0,
  "beforeordercreation": 0,
  "beforepartialrollbacks": 0,
  "aftercommit": 0,
  "afterrollback": 0,
  "afterpaymentcapture": 0,
  "afterinventorydeductions": 0,
  "afterordercreation": 0,
  "afterpartialrollbacks": 0,
  "tasksetid": "msc-37788-01:38-65563392",
  "explanation": "A valid completed checkout must have exactly one order, one captured payment, and one inventory deduction. Before mode leaves partial side effects. After mode /commit() atomically rolls all side effects back."
}
```
- Summary Table:**

Code	Description	Links
200	OK	No links

المؤشرات النهائية للـ transaction integrity.

Requirement 09 - Stress Testing 100 Concurrent Users .6



مخطط معماري: اختبار الضغط عبر JMeter و endpoint داخلي لـ 100 مستخدم.

6.1 المشكلة الهندسية

المطلوب ليس أن تعمل العمليات في حالة مستخدم واحد فقط، بل أن يثبت النظام قدرته على خدمة 100 مستخدم متزامن دون انهيار أو فقدان بيانات. هذا يعني أن الاختبار يجب أن يغطي عمليات متعددة وليس endpoint بسيطاً فقط.

6.2 لماذا استخدمنا JMeter + Swagger ؟

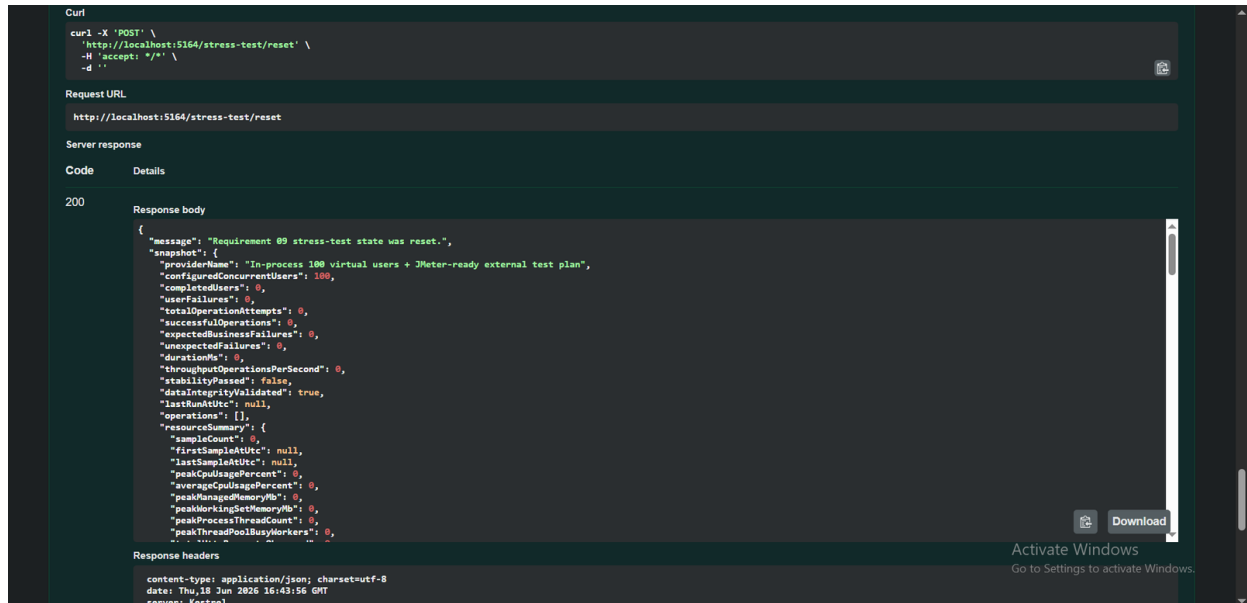
الأداة	دورها	سبب الاختيار
Swagger internal stress endpoint	تشغيل 100 مستخدم افتراضي داخل التطبيق مع فحص invariants	سهل العرض ويعطي proof منطقي
JMeter	ضغط خارجي حقيقي عبر HTTP وقياسات Summary/Aggregate	مناسب جداً لمتطلب الدكتور وملف jmx.
k6/wrk	أدوات قوية للضغط	لم نختار لأن JMeter أسهل للتسليم بالصور والـ GUI
اختبار يدوي فقط	غير كافٍ	لا يثبت 100 concurrent users

6.3 المشاكل التي واجهتنا

- التمييز بين business failures المقبولة والفشل الحقيقي في النظام. مثال: نفاد الكوبون أو نفاد المخزون ليس انهياراً بل نتيجة تجارية صحيحة.
- تشغيل Redis وقاعدة البيانات قبل الاختبار حتى لا تظهر failures غير مرتبطة بالأداء.
- التأكد أن JMeter يستخدم port الصحيح 5164 وأن الـ protocol هو http.
- اختيار ramp-up مناسب حتى لا يكون الاختبار غير واقعي أو شديد الانفجار على الجهاز المحلي.

6.4 نتائج Swagger

تم تشغيل endpoint داخلي لـ 100 مستخدم. الصورة تظهر `unexpectedFailures = 0`، `completedUsers = 100`، `stabilityPassed = true`، و `dataIntegrityValidated = true`. هذا يعني أن النظام تعامل مع الضغط ولم تظهر أخطاء غير متوقعة أو تخريب بيانات.



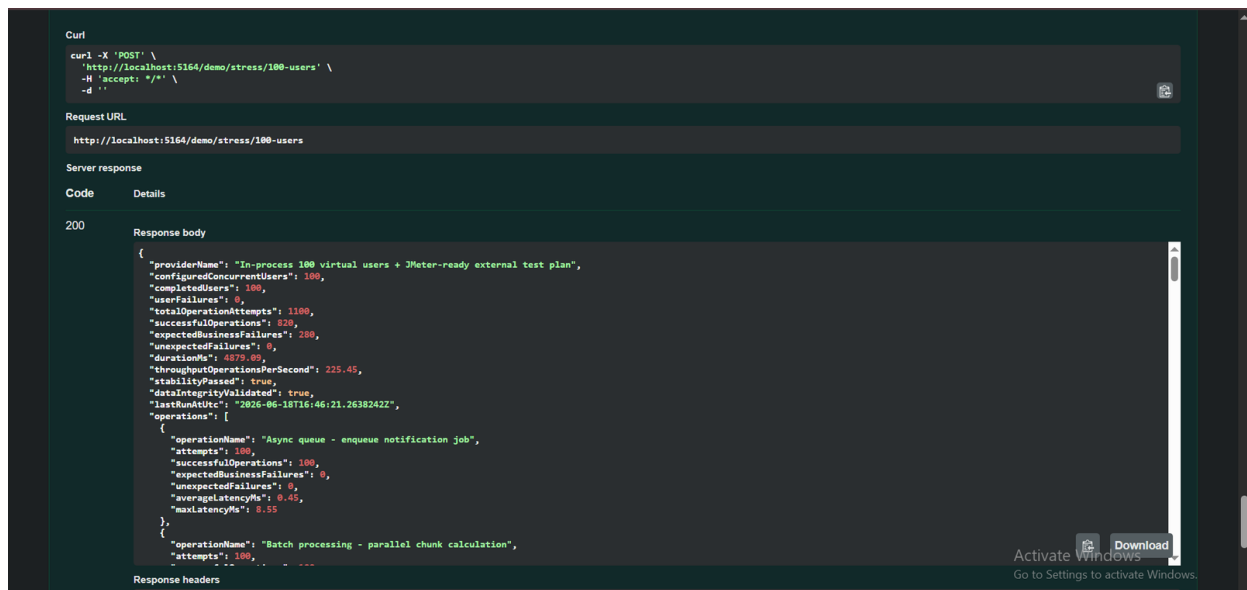
```
curl -X 'POST' \
  'http://localhost:5164/stress-test/reset' \
  -H 'accept: */*' \
  -d ''

Request URL
http://localhost:5164/stress-test/reset

Server response
Code    Details
200

Response body
{
  "message": "Requirement 09 stress-test state was reset.",
  "snapshot": {
    "providerName": "In-process 100 virtual users + JMeter-ready external test plan",
    "configuredConcurrentUsers": 100,
    "completedUsers": 0,
    "userFailures": 0,
    "totalOperationAttempts": 0,
    "successfulOperations": 0,
    "expectedBusinessFailures": 0,
    "unexpectedFailures": 0,
    "durationMs": 0,
    "throughputOperationsPerSecond": 0,
    "stabilityPassed": false,
    "dataIntegrityValidated": true,
    "lastRunAtUtc": null,
    "operations": [],
    "resourceSummary": {
      "sampleCount": 0,
      "firstSampleAtUtc": null,
      "lastSampleAtUtc": null,
      "peakCpuUsagePercent": 0,
      "averageCpuUsagePercent": 0,
      "peakManagedMemoryMb": 0,
      "peakWorkingSetMemoryMb": 0,
      "peakProcessThreadCount": 0,
      "peakThreadPoolBusyWorkers": 0
    }
  }
}
```

إعادة ضبط حالة اختبار الضغط قبل التشغيل.



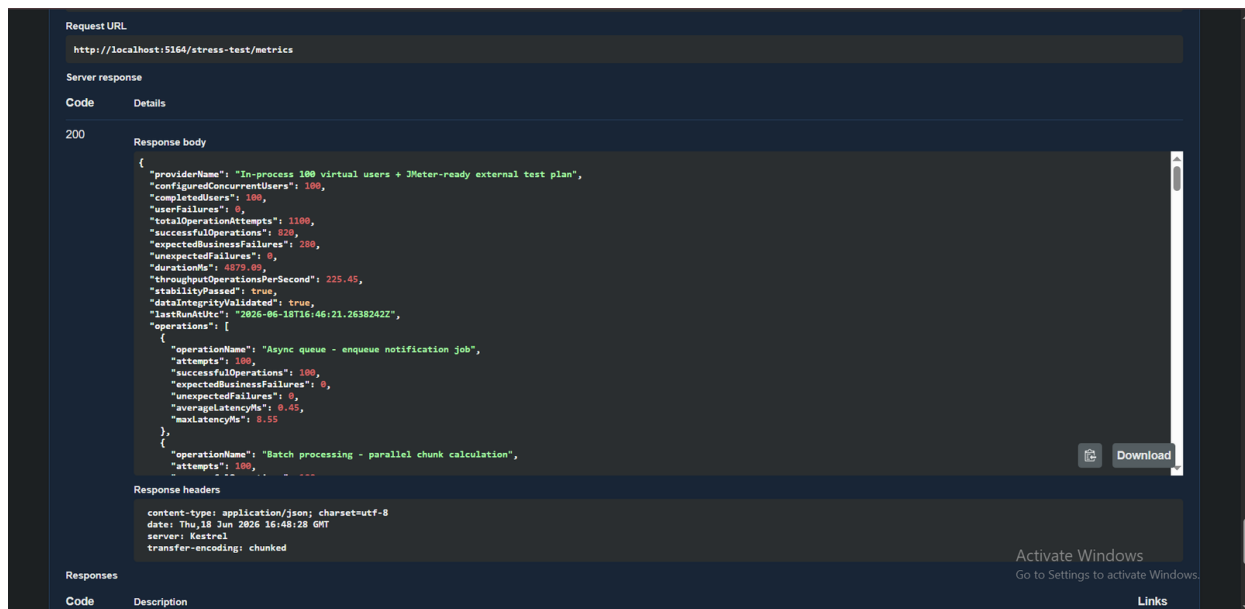
```
curl -X 'POST' \
  'http://localhost:5164/demo/stress/100-users' \
  -H 'accept: */*' \
  -d ''

Request URL
http://localhost:5164/demo/stress/100-users

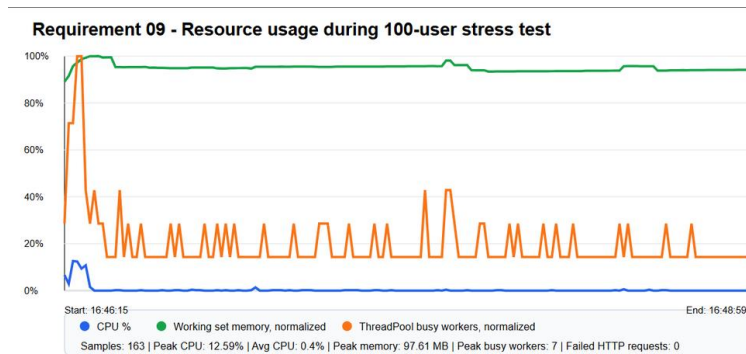
Server response
Code    Details
200

Response body
{
  "providerName": "In-process 100 virtual users + JMeter-ready external test plan",
  "configuredConcurrentUsers": 100,
  "completedUsers": 100,
  "userFailures": 0,
  "totalOperationAttempts": 1100,
  "successfulOperations": 520,
  "expectedBusinessFailures": 230,
  "unexpectedFailures": 0,
  "durationMs": 4879.09,
  "throughputOperationsPerSecond": 225.45,
  "stabilityPassed": true,
  "dataIntegrityValidated": true,
  "lastRunAtUtc": "2026-06-18T16:46:21.263824Z",
  "operations": [
    {
      "operationName": "Async queue - enqueue notification job",
      "attempts": 100,
      "successfulOperations": 100,
      "expectedBusinessFailures": 0,
      "unexpectedFailures": 0,
      "averageLatencyMs": 0.45,
      "maxLatencyMs": 0.55
    },
    {
      "operationName": "Batch processing - parallel chunk calculation",
      "attempts": 100,
      "successfulOperations": 100,
      "expectedBusinessFailures": 0,
      "unexpectedFailures": 0,
      "averageLatencyMs": 0.45,
      "maxLatencyMs": 0.55
    }
  ]
}
```

نتيجة 100: Swagger مستخدم، نجاح الاستقرار وسلامة البيانات.



المؤشرات النهائية لاختبار الضغط.



رسم موارد النظام أثناء/بعد اختبار الضغط.

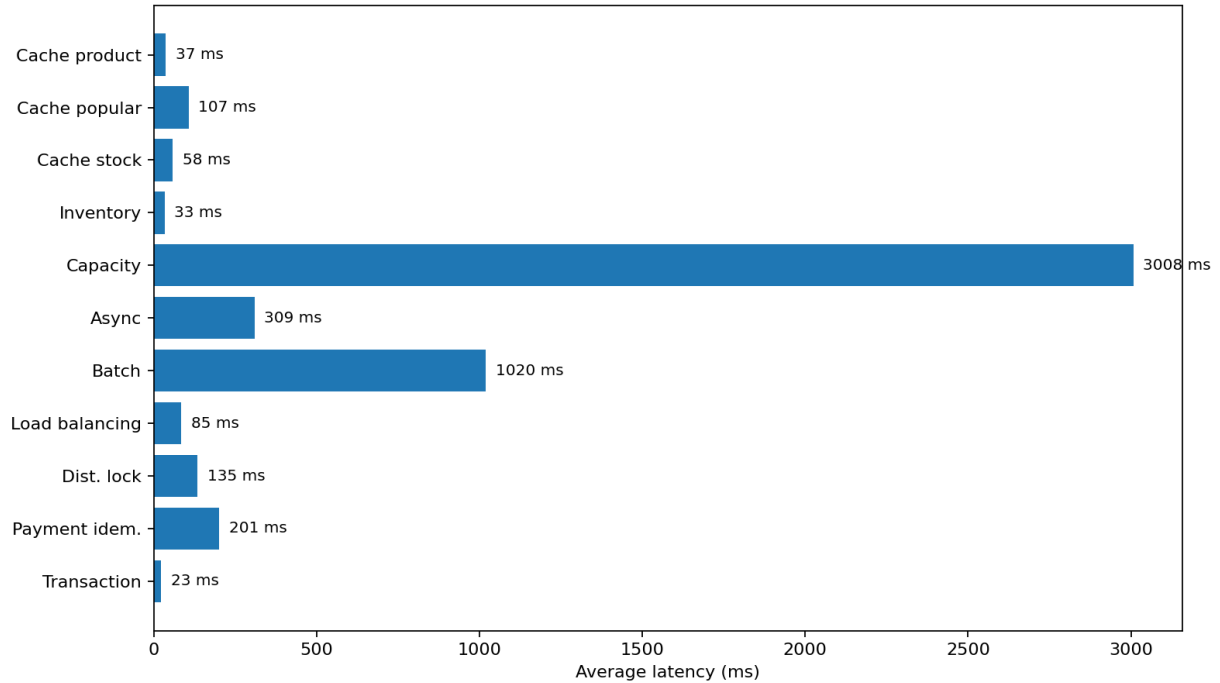
6.5 نتائج JMeter

تم تشغيل ملف Requirement09_Stress100Users_AllOperations.jmx. النتائج أظهرت 1101 عينة، Error = 0.00% ، ومتوسط كلي 456ms وجود Error = 0.00% هو الدليل الأهم أن الاختبار مر بدون فشل HTTP أو انهيار.

المؤشر	القيمة	الدلالة
Total Samples	1101	الاختبار لم يقتصر على طلب واحد؛ بل غطى عدة عمليات
Average	456 ms	متوسط زمن الاستجابة الكلي
Maximum	5511 ms	أعلى زمن بسبب عملية-capacity controlled الثقيلة

المؤشر	القيمة	الدلالة
Error %	0.00%	لا توجد أخطاء HTTP في JMeter
Throughput	88.8/sec	معدل تنفيذ قوي على جهاز محلي

Requirement 09 - JMeter Average Latency by Operation



مخطط من JMeter: متوسط زمن الاستجابة لكل عملية. يظهر أن capacity و batch هما الأكثر تكلفة، وهذا منطقي بسبب طبيعة العمل.

Requirement09_Stress100Users_AllOperations.jmx (D:\fourthYear\second\ParallelProgramming\ParallelECommerce_Req10\Tests\JMeter\Requirement09_Stress100Users_AllOperations.jmx) - Apache...

File Edit Search Run Options Tools Help

00:00:13 0 0/100

Requirement09 - 100 Users All Operations Stress Test

- HTTP Request Defaults
- HTTP Header Manager
- Setup - Reset system state
- POST /stress-test/reset
- 100 concurrent users - all operations
- Summary Report
- Aggregate Report
- View Results Tree

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST /stres...	1	184	184	184	0.00	0.00%	5.4/sec	29.31	1.10	5523.0
GET cache p...	100	37	4	120	42.20	0.00%	20.2/sec	9.52	3.79	482.3
GET cache p...	100	107	4	200	69.21	0.00%	20.2/sec	13.02	4.06	659.8
GET cache s...	100	58	4	141	27.24	0.00%	20.7/sec	10.49	3.81	519.6
POST invent...	100	33	15	122	22.57	0.00%	20.9/sec	7.06	4.75	346.0
POST capac...	100	3008	529	5511	1471.93	0.00%	9.9/sec	5.33	2.01	554.1
POST async ...	100	309	298	325	6.15	0.00%	10.1/sec	10.35	2.78	1050.7
POST batch ...	100	1020	967	1152	42.69	0.00%	9.4/sec	4.83	1.89	525.5
POST load b...	100	85	72	109	9.08	0.00%	10.3/sec	3.98	2.18	396.7
POST distrib...	100	135	26	337	73.68	0.00%	10.2/sec	3.99	2.70	402.7
POST idemp...	100	201	24	1746	434.26	0.00%	10.2/sec	4.51	3.01	451.1
POST transa...	100	23	2	159	46.27	0.00%	12.4/sec	13.81	4.72	1137.8
TOTAL	1101	456	2	5511	969.83	0.00%	88.8/sec	51.82	21.24	597.8

☐ Include group name in label? ☒ Save Table Header

JMeter Summary Report: 1101 samples. Error = 0.00%.

Requirement09_Stress100Users_AllOperations.jmx (D:\fourthYear\second\ParallelProgramming\ParallelECommerce_Req10\Tests\JMeter\Requirement09_Stress100Users_AllOperations.jmx) - Apache...

FileEditSearchRunOptionsToolsHelp

Requirement09 - 100 Users All Operations Stress Test

HTTP Request Defaults

HTTP Header Manager

Setup - Reset system state

POST /stress-test/reset

100 concurrent users - all operations

Summary Report

Aggregate Report

View Results Tree

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

Errors

Successes

Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughp...	Received ...	Sent KB/s...
POST /str...	1	184	184	184	184	184	184	184	0.00%	5.4/sec	29.31	1.10
GET cach...	100	37	8	100	104	117	4	120	0.00%	20.2/sec	9.52	3.79
GET cach...	100	107	150	160	164	184	4	200	0.00%	20.2/sec	13.02	4.06
GET cach...	100	58	63	74	103	124	4	141	0.00%	20.7/sec	10.49	3.81
POST inv...	100	33	24	61	90	114	15	122	0.00%	20.9/sec	7.06	4.75
POST cap...	100	3008	3023	5111	5294	5490	529	5511	0.00%	9.9/sec	5.33	2.01
POST asy...	100	309	308	317	320	324	298	325	0.00%	10.1/sec	10.35	2.78
POST bat...	100	1020	1014	1032	1145	1151	967	1152	0.00%	9.4/sec	4.83	1.89
POST loa...	100	85	83	95	104	109	72	109	0.00%	10.3/sec	3.98	2.18
POST dist...	100	135	129	232	259	288	26	337	0.00%	10.2/sec	3.99	2.70
POST ide...	100	201	33	691	1242	1721	24	1746	0.00%	10.2/sec	4.51	3.01
POST tran...	100	23	3	113	143	155	2	159	0.00%	12.4/sec	13.81	4.72
TOTAL	1101	456	89	1032	2720	5005	2	5511	0.00%	88.8/sec	51.82	21.24

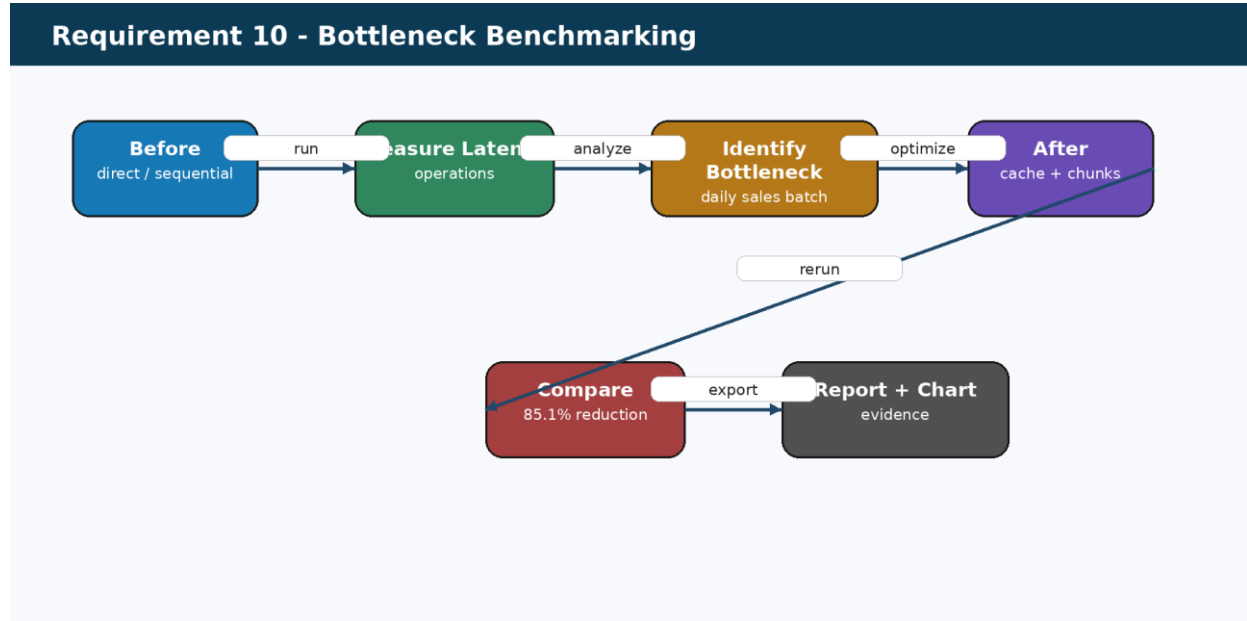
Include group name in label?

Save Table Data

Save Table Header

JMeter Aggregate Report: percentiles لى عملية.

Requirement 10 - Bottleneck Analysis & Benchmarking .7



مخطط معماري: قياس before/after ثم تحديد الاختناق وتقديم تقرير.

7.1 المشكلة الهندسية

الطلب العاشر لا يطلب تحسينًا عامًا فقط، بل يطلب قياس زمن الاستجابة لأهم العمليات، تحديد bottleneck واحد على الأقل، ثم تقديم مقارنة رقمية قبل التحسين وبعده. لذلك كان لا بد من بناء benchmark قابل للتكرار داخل المشروع.

7.2 لماذا استخدمنا Benchmark داخلي؟

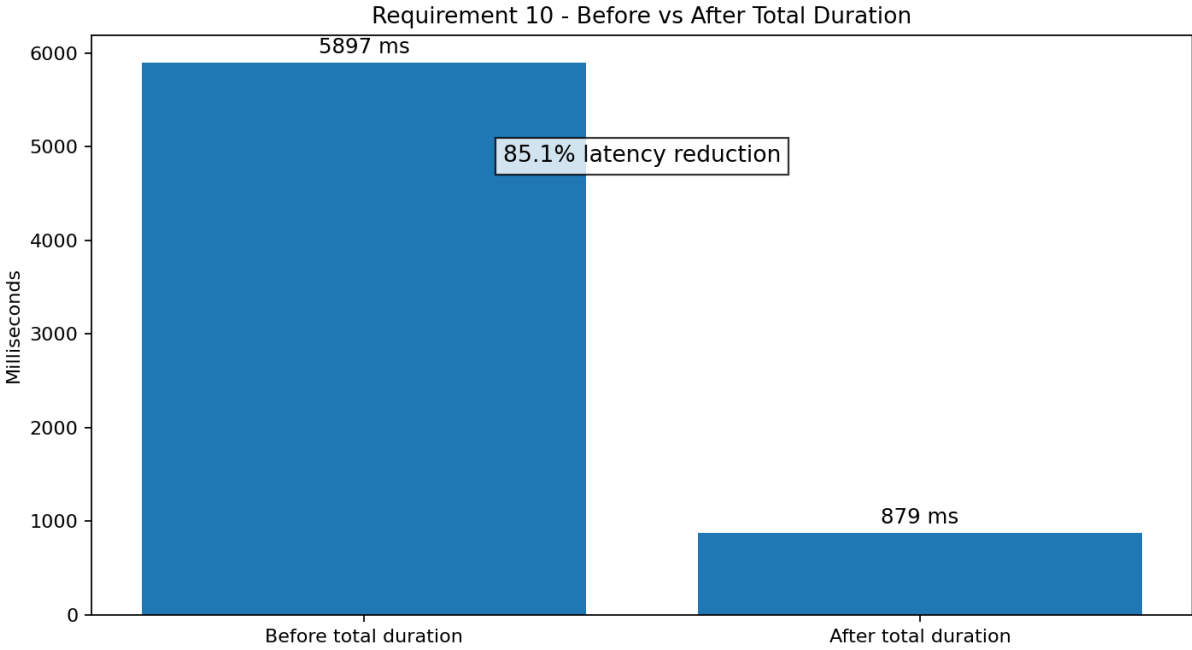
الخيار	السبب	القرار
قياس يدوي بالساعة أو Swagger فقط	غير دقيق ولا يعطي مقارنة منظمة	مرفوض
Profiler خارجي	مفيد لكنه قد لا يكون متاحًا في جهاز الدكتور	ليس الخيار الأساسي
APM مثل Prometheus/Grafana	احترافي لكنه يزيد التعقيد والإعدادات	أكبر من نطاق المشروع
Benchmark endpoint داخلي	قابل للتشغيل من Swagger ويعطي JSON واضح وصور	تم اختياره

7.3 الاختناق المحدد

الاختناق المحدد في النتيجة هو Daily sales batch calculation. السبب الجذري أن المعالجة التسلسلية تجعل الزمن الكلي ينمو خطيًا مع عدد السجلات. الحل التطبيقي كان Parallel chunk processing with bounded concurrency، أي تقسيم البيانات إلى chunks ومعالجتها بالتوازي ضمن حد آمن

7.4 نتائج القياس

المؤشر	القيمة	الدلالة
BeforeTotalDurationMs	5897.08 ms	الوضع قبل التحسين كان بطيئاً بسبب المعالجة التسلسلية
AfterTotalDurationMs	878.67 ms	الوضع بعد التحسين أسرع بوضوح
OverallLatencyReductionPercent	85.1%	تحسن رقمي قوي
IdentifiedBottleneck	Daily sales batch calculation	تم تحديد bottleneck بوضوح
BenchmarkPassed	true	التحسين حقق معيار القبول



مخطط رقمي: قبل التحسين وبعده في الطلب 10.

```
Curl
curl -X 'POST' \
  'http://localhost:5164/benchmark/reset' \
  -H 'accept: */*' \
  -d ''

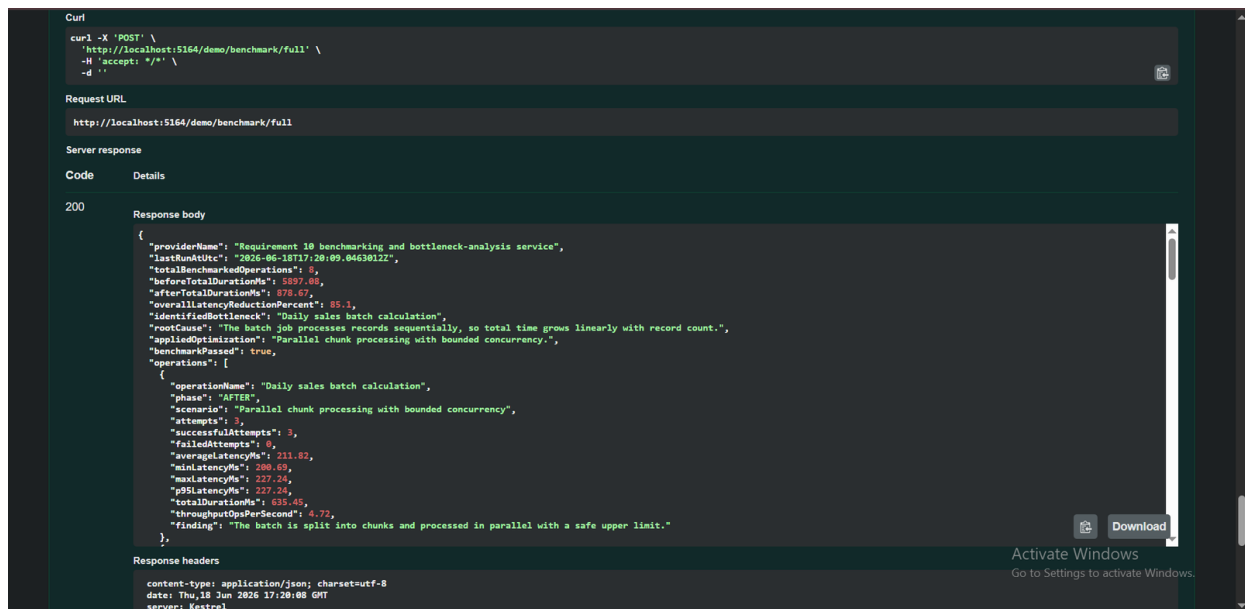
Request URL
http://localhost:5164/benchmark/reset

Server response
Code    Details
200

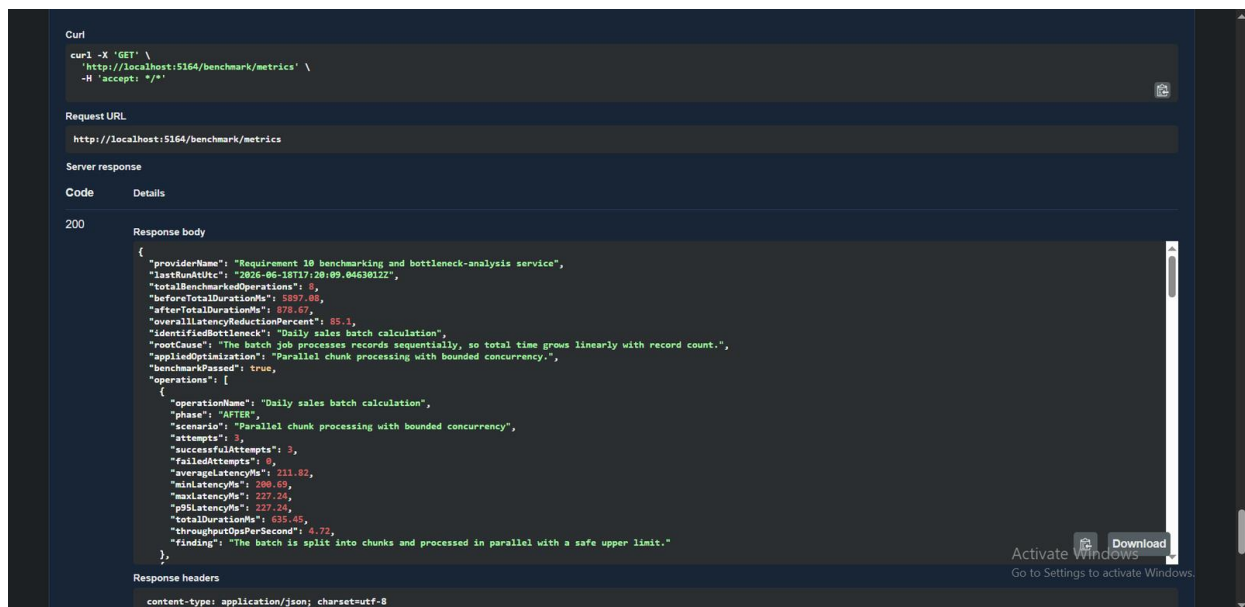
Response body
{
  "message": "Requirement 10 benchmark state was reset.",
  "snapshot": {
    "providerName": "Requirement 10 benchmarking and bottleneck-analysis service",
    "testRunTitle": null,
    "totalBenchmarkedOperations": 0,
    "beforeTotalDurationMs": 0,
    "afterTotalDurationMs": 0,
    "overallLatencyReductionPercent": 0,
    "identifiedBottleneck": "Not identified yet",
    "rootCause": "Run POST /demo/benchmark/full to measure baseline and optimized operations.",
    "appliedOptimization": "Not applied yet",
    "benchmarkPassed": false,
    "operations": [],
    "comparisons": [],
    "resourceSummary": {
      "sampleCount": 0
    },
    "explanation": "Requirement 10 requires numeric before/after benchmarking and at least one identified bottleneck."
  }
}

Response headers
content-type: application/json; charset=utf-8
date: Thu, 18 Jun 2026 17:19:13 GMT
server: Kestrel
transfer-encoding: chunked
```

إعادة ضبط benchmark قبل القياس.



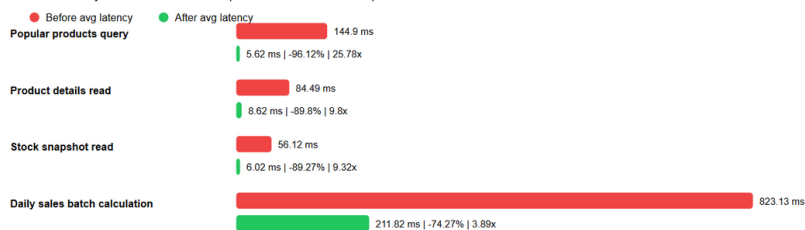
نتيجة benchmark: full تحديد الاختناق وتحسن 85.1%



المؤشرات النهائية للـ benchmark

Requirement 10 - Before/After Benchmark

Bottleneck: Daily sales batch calculation | Overall reduction: 85.1% | Passed: True



Before total: 5897.08 ms | After total: 878.67 ms | Root cause: The batch job processes records sequentially, so total time grows linearly with record count.

رسم قبل/بعد للعمليات المقاسة.

```
# Requirement 10 - Bottleneck Analysis & Benchmarking

- Benchmark passed: **True**
- Identified bottleneck: **Daily sales batch calculation**
- Root cause: The batch job processes records sequentially, so total time grows linearly with record count.
- Applied optimization: Parallel chunk processing with bounded concurrency.
- Before total duration: 5897.08 ms
- After total duration: 878.67 ms
- Overall latency reduction: 85.1%

| Operation | Before avg ms | After avg ms | Reduction % | Speedup | Optimization |
|---|---|---|---|---|---|
| Popular products query | 144.9 | 5.62 | 96.12 | 25.78x | Redis caching for the read-heavy popular-products result. |
| Product details read | 84.49 | 8.62 | 89.8 | 9.8x | Redis cache-aside with invalidation on stock-changing operations. |
| Stock snapshot read | 56.12 | 6.89 | 87.97 | 8.14x | Short-lived Redis stock snapshot with invalidation when stock changes. |
| Daily sales batch calculation | 823.13 | 211.82 | 74.27 | 3.89x | Parallel chunk processing with bounded concurrency. |

Conclusion:
The benchmark measured key operations before and after optimization, identified a bottleneck, and showed numeric latency improvement.
```

Activate Windows
Go to Settings to activate Windows.

تقرير Markdown generated من endpoint /benchmark/report.

8. مصفوفة الأدلة النهائية

الحكم	JMeter Evidence	Swagger Evidence	المتطلب
ناجح	Requirement06_Caching.jmxيفضل	Req06_06إلىReq06_09	06
ناجح بقوة	Distributed Lockاختياري للنسخة القديمة	Req07_01_RealDb_Reset + Req07_02_RealDb_OptimisticLocking	07
ناجح بقوة	Transaction Integrityاختياري للنسخة القديمة	Req08_01_RealDb_Reset + Rollback + Commit	08
ناجح	Summary + Aggregateمع0% error	Req09_04إلىReq09_01	09
ناجح	يفضل Requirement10_BottleneckBenchmarking.jmx	Req10_05إلىReq10_01	10

9. خطة إعادة الاختبار النهائية

قبل إعادة الاختبار يجب تشغيل المكونات الخارجية LocalDB: للطلبات 7 و8، و Redis للطلب 6 وأجزاء من الضغط وال benchmark. الأوامر المقترحة:

```
sqllocaldb start MSSQLLocalDB
```

```
dotnet ef database update
```

```
docker compose -f docker-compose.cache.yml up -d
```

```
docker exec -it parallel-ecommerce-redis redis-cli ping
```

```
dotnet run
```

9.1 مسار Swagger المختصر

المتطلب	الأوامر الأساسية من Swagger
06	POST /demo/cache/before ثم POST /cache/reset GET /cache/metrics ثم POST /demo/cache/after
07	POST /demo/db-ثم POST /db-concurrency/reset concurrency/stock/after
08	POST /db-ثم POST /db-transaction/reset commit: rollback مرتين transaction/checkout/after
09	POST /demo/stress/100-ثم POST /stress-test/reset /resources/svg و GET /stress-test/metrics ثم users
10	POST ثم POST /benchmark/reset و GET /benchmark/report ثم demo/benchmark/full /benchmark/chart/svg

